Garbling a Hash-Based STARK Verifier for Bitcoin

Stephen Duan*[1, 2], Vanhger Pham[1], Blake Hou[1], Ethan Zhu[2], and Jingping Yin[1]

[1]ZKM Team
[2]GOAT Network

Abstract

BitVM-style protocols verify succinct proofs on Bitcoin by garbling a proof verifier off-chain. Existing designs check elliptic-curve SNARKs, which a quantum adversary breaks and which need a trusted or secret setup. A hash-based STARK verifier removes that dependency but is hard to garble: it branches on data, its transcript is fixed by an implementation, hashing and field multiplication dominate, and a stored circuit of 10^8 gates does not fit in memory. We garble the Plonky3 verifier of a Keccak-f algebraic intermediate representation (AIR) without public values over WHIR, in the binary tower field $\mathbb{F}_{2^{128}}$ with Blake3 throughout, using free-XOR gadgets and a streaming garbler that holds one label per live wire. For a 2^{18}-row trace at 104 bits of soundness, the verifier has 94.1M non-free gates and garbles to 1.5 GB in under a minute on one core, about 30× fewer non-free gates than the Boolean-garbled Groth16 verifier of a different statement. The dominant Bitcoin-side cost is the 1,041,024 input bits: Bitcoin Core-tested fixtures that reveal and authenticate them for one key set measure 2.22 million virtual bytes (MvB) with Schnorr adaptors and 7.86–17.27 MvB with hash-based one-time keys, against a few kilobytes for Groth16. These are measured components, not a total on-chain cost: the dispute graph and the soldering of one key set to the kept garblings are not constructed. The proof system gives 104 classical and about 52 quantum bits under stated assumptions, while cut-and-choose gives about 40 bits and Bitcoin's signatures are not post-quantum. Recursing a zkVM proof through a narrow tape machine, with unreviewed proof-system changes that we list, brings the input to 564,552 bits in 218.0M non-free gates, without binding the zkVM program or its outputs.

1 Introduction

Bitcoin cannot verify a succinct proof economically within current Script and policy limits, but a trust-minimized bridge needs such verification. Consider a withdrawal from a rollup: an operator has paid out a user on Bitcoin and must prove, to a Bitcoin script, that the rollup's state authorizes the payment; the proof is a STARK over the rollup's execution. This bridge is the motivating application. The implementation evaluated in this paper verifies a Keccak-f AIR without public values, so it does not yet bind a proof to a withdrawal. Hashing dominates such statements, and we evaluate on a hash-heavy standard case, proofs of Keccak-f permutations [Nat15].

A SNARK verifier written in Script takes hundreds of megabytes [GKS+26]. The BitVM line of work therefore verifies *optimistically*: a prover claims a result, and a challenger disputes it only if it is wrong. BitVM2 [LAA+25] settles a dispute by bisecting the verifier's execution trace on-chain, which is secure but expensive: a mainnet experiment put the unhappy path at over

*Corresponding author: `stephen.d@zkm.io`

$14,000 [GKS+26]. BitVM3 moves the verifier off-chain into a *garbled circuit* [Lin25, WAA+26, Che25]. At setup the verifier circuit is garbled. At dispute time the proof is published under one-time signatures that give the evaluator the circuit's input labels, and the outcome depends on which output label the evaluation yields. For the Groth16-sized instances reported by BitVM3 and BABE, only the signed proof and a hashlock remain on-chain, for tens of dollars [GKS+26].

The off-chain side is where the existing designs fall short. A garbled circuit has to be generated, stored for the lifetime of the deposit and checked, because the garbler may cheat, typically by cut-and-choose over many instances. The Boolean-garbled Groth16 verifier [Bit25] has about 3×10^9 non-free gates, some 42 GiB, and hours of cut-and-choose setup [GKS+26]. Garbling Groth16 with native group operations [KFLT26] and witness encryption [GKS+26] shrink the artifact to megabytes. All of these designs, however, verify an elliptic-curve SNARK, pairing-based or its designated-verifier form over BN254 or a binary curve: a quantum adversary breaks them, and they need a trusted or secret setup. A STARK proof must first be wrapped in such a SNARK, which reintroduces both.

Key idea. We garble a hash-based STARK verifier directly and make it cheap by choosing its field. Under free-XOR [KS08] a garbled circuit pays only for non-linear gates, and in a field of characteristic two every addition is an XOR. A STARK over the binary tower field $\mathbb{F}_{2^{128}}$ with Blake3 commitments therefore costs, as a garbled circuit, its hashes and field multiplications, with additions free. Its algebraic checks rest on WHIR over $\mathbb{F}_{2^{128}}$, while Blake3 instantiates the commitments, the Fiat–Shamir transcript and the garbling hash, modeled as circular correlation robust (§7).

Challenges. Three problems stand between this idea and a working garbled verifier.

C1 *A fixed circuit for a branching verifier.* A garbled circuit has one shape for all inputs, while the verifier redraws challenges and fixes coordinates depending on sampled values.

C2 *A transcript defined only by code.* The circuit must absorb and sample exactly the bytes that the prover's implementation does, or Fiat–Shamir soundness is lost, and that byte schedule is specified nowhere outside the implementation.

C3 *Hashing is expensive.* Merkle paths, leaf rows and the transcript are Blake3 compressions, and a compression costs more than four field multiplications.

A fourth, engineering obstacle is memory: a verifier of 10^8 non-free gates has several hundred million wires, and a builder that stores the gate list needs tens of gigabytes before garbling starts. A streaming garbler, which counts each wire's uses in a plan pass and releases a wire's slot after its last use, removes it (§6).

Contributions. The first two contributions answer C1–C3; the third integrates the verifier with Bitcoin.

- **A fixed-shape circuit for the measured verifier path (C1, C2).** We implement the path through Plonky3's STARK verifier [Plo26] exercised by our Keccak-f AIR over the Boolean WHIR trace commitment as a Boolean circuit with one output wire: the transcript, the zerocheck of the AIR, the batching of the opened columns, the bit ring switch and the WHIR opening. Data-dependent branches are fixed and guarded, and the transcript follows a byte schedule recovered from Plonky3's own verifier, so one circuit serves every proof of a configuration and can be garbled before the proof exists (§4).

- **Cheaper gadgets (C3).** Tower-field multiplication at 3^k AND gates per 2^k bits, with squaring and multiplication by the tower generator free, and a Blake3 circuit at 10,281 AND gates per compression instead of 20,657, with tree mode for long inputs (§4.2).
- **Bitcoin integration.** We place the verifier in a BitVM3-style dispute, state the label-selection task that publishing its input must solve, and price four one-time input schemes and two transport baselines as complete signed transactions replayed through Bitcoin Core, separating the measured one-key-set input-authentication component from the unimplemented multi-copy soldering and transaction graph (§5). Including signature or preimage witness data and authentication scripts, this component measures 2.22 MvB with the 8-bit adaptor baseline and 7.86–17.27 MvB per hash-key set, against a few kilobytes for Groth16. These are not total on-chain footprints. Tuning removes only a fifth of the logical input.

Alongside these, §7.1 analyzes the STARK and garbling components in the quantum random-oracle model, including Grover search on the garbling offset, and separates those bounds from the lower cut-and-choose level and the non-post-quantum transaction layer; §6 measures real Keccak-f proofs from 2^5 to 2^{20} rows, profiles and ablates the circuit, compares it with pairing-based designs and estimated garbled STARK verifiers, and garbles the verifier of a recursion proof of Ziren's binary stage at 564,552 input bits. The circuit builder and the measurements are open source; the streaming garbler, the Blake3 circuit and a label-level evaluator are contributed to GOAT's bitvm-gc [GOA26b].

2 Related work

We group prior work by how it verifies a proof on Bitcoin: by garbling a verifier, by witness encryption, or through small pairing-based SNARKs that both approaches rely on. We close with the garbling techniques we use.

BitVM and garbled verifiers. BitVM2 [LAA+25] verified optimistically by bisecting the verifier's trace on-chain, and BitVMX [LAM+24] bisected the trace of a general CPU in the same way. ColliderVM [KLN25] left the optimistic model: it placed the computation on-chain in validity form and passed state between transactions through hash-collision puzzles, without fraud proofs, at an on-chain cost that its authors did not claim to be viable. Delbrag [Rub25] first proposed replacing BitVM's fraud proof by the off-chain evaluation of a garbled circuit whose outcome is revealed as a secret, at the price of more off-chain exchange and of expensive proofs of circuit correctness. BitVM3 took up that idea and garbled the verifier [Lin25, Che25], and its formal treatment defined BitVM3-CORE as a sound and complete on-chain proof system with on-chain costs of about $9 [WAA+26]. The Boolean-garbled Groth16 verifier [Bit25] has about 3×10^9 non-free gates, 42 GiB, with cut-and-choose setup of over three hours at $(N, M) = (181, 7)$ [GKS+26]. Garbling Groth16 with native elliptic-curve group operations reduced the garbled program to 2.4 MiB [KFLT26]. Glock [Eag25] formalized garbled locks and garbled a designated-verifier variant of Pari; BABE reports its garbled circuit at about 12 MB on a binary elliptic curve [GKS+26]. GOAT's bitvm-gc [GOA26b] garbles Groth16 and a designated-verifier form of Pari [DMS24] over BN254 and over the binary curve sect233k1. We garble a measured STARK-verifier path with the same garbling code, and we contribute our streaming garbler and Blake3 circuit back to it. BitVM3's comparison of garbling schemes already lists a Yao-style garbled STARK verifier, at about 10 GB of off-chain storage and as an estimate [WAA+26, Table 9]; we build one and measure 1.5 GB per instance. TRAPGC-DV [FBF+26] verified Groth16 by designated evaluation: the prover wraps its proof in an outer

Groth16 proof under a per-session trapdoor SRS and garbles a small circuit that checks it in scalar form, with cut-and-choose and a universal-composability analysis. It too rests on pairings.

Garbling on Bitcoin: security and inputs. Futoransky, Larotonda and Barbara [FLB25] broke the authenticity of the RSA-based garbling proposed for BitVM3 [Lin25], of the label forward-propagation variant [FDZ25], and of linear adaptors: coprime public exponents let the evaluator forge labels by extended Euclid, and linear adaptors fall to the Franklin–Reiter related-message attack. Our hash-based garbling has no such label algebra. Mosaic [KTA+26] made cut-and-choose maliciously secure with an on-chain footprint independent of the number of garbled copies, and Antichain Winternitz [TF26a] published input labels on-chain at a cost up to 52.9% lower than Lamport signatures. Both are independent of the garbled circuit; §5 prices both for our million-bit input. Tiwari and Feickert [TF26b] assessed the post-quantum cost of such bridges, estimated Boolean STARK verifier sizes, and found proof size the main obstacle to on-chain feasibility.

Quantum-safe transaction authorization. QSB [Lev26] constructed a hash-based authorization path under existing legacy-Script consensus rules by combining commitments in the style of HORS (hash to obtain a random subset) with transaction-bound hash-to-signature grinding. A QSB spend was mined on Bitcoin mainnet in August 2026 [Sta26]. The construction is non-standard and requires direct miner inclusion. It authorizes a specially constructed spend; adapting it to a multi-transaction, pre-signed and timelocked dispute graph remains open.

Verifying a STARK in Script. Our companion work, in an earlier version of this paper's repository, verifies the same kind of proof directly in Bitcoin Script, with Poseidon2 over KoalaBear and WHIR and without `OP_CAT` [GOA26a]. It needs no garbling and no trusted party, but its 2^{20} example configuration requires 1.24 GB of script, equivalent to 309 blocks; moreover, a standard transaction holds less than one Poseidon2 permutation, so it must be split into sub-query steps. Garbling moves that cost off-chain.

Witness encryption on Bitcoin. BABE [GKS+26] replaced the garbled Groth16 verifier with witness encryption for linear pairing relations [GKPW24]. It made the one quadratic pairing $e(\pi_1, \pi_2)$ linear by computing $r\pi_1$ in a small garbled circuit, built from decomposable randomized encodings on top of Argo MAC [EL26]. The per-instance artifact is 22 MiB, the on-chain cost is comparable to BitVM3, and cut-and-choose setup takes seconds. Deferred binding [Pha26] extended BABE to public inputs known only at dispute time: a second garbled circuit computes a blinded point $Q = x_D L_D + B$ from the late input and a third computes rQ, relying on Groth16's linear public-input aggregation. Its implementation reported 0.92 GiB per finalized instance, 768 input labels, and an Assert of 2,154 bytes with a 14,312-byte ChallengeAssert witness; §6 quotes these figures.

Witness encryption for covenants. Bitcoin PIPEs v2 [ACG+26] witness-encrypted a Schnorr signing key under an NP statement to emulate covenants without a soft fork. They defined witness signatures and mapped BitVM's disprove condition to a relation $f(w) = 0 \wedge h = \mathrm{Hash}(w)$, which requires the witness at setup. Their witness encryption comes from arithmetic affine determinant programs [SRG+26], an arithmetic variant of affine determinant programs [BIJ+20]. The ciphertext grows as $v(2g+1)^2$ field elements, cubic in the constraint count, which is about 338 TB for a purpose-built 14,371-constraint pairing-SNARK verifier. Security is heuristic: an earlier version fell to a rank-drop attack, and the linearization analysis is described as inconclusive. All witness-encryption

designs above are smaller off-chain than a garbled verifier, but all require the statement to be a pairing SNARK, so a STARK would first have to be wrapped in one, and none is post-quantum.

Small SNARK verifiers. Pari [DMS24] gave the smallest known pairing-based proofs, two $\mathbb{G}_1$ and two field elements, with a three-pairing verifier; its equifficient polynomial commitments also gave Garuda, a prover with free linear gates. Pari underlies bitvm-gc's designated-verifier SNARK, Glock, and the SNARK of the AADP construction; Glock and bitvm-gc's `sect233k1` verifier use its designated-verifier form on a binary curve, which needs no pairing but still rests on discrete logarithms. Groth16 [Gro16] remains the deployed baseline.

Binary-field proof systems. Binius [DP25] built succinct arguments over towers of binary fields, and ring switching [DP26] turned a claim about a polynomial's bits into one about its packed extension-field form; Plonky3's Boolean WHIR commitment, which we garble, uses both ideas with WHIR [ACFY25] as the proximity test. Its cheap additions are what make a garbled verifier small.

Garbling. We use privacy-free garbling [FNO15] with free-XOR [KS08], one ciphertext per AND gate [ZRE15], and the syntax of Bellare, Hoang and Rogaway [BHR12b]; free-XOR's security rests on circular correlation robustness [CKKZ12]. Witness encryption in general goes back to Garg, Gentry, Sahai and Waters [GGSW13].

3 Setting

3.1 Garbled verifiers on Bitcoin

We follow the BitVM3 setting [WAA+26]. A *garbler*, the operator who proves, and an *evaluator*, a challenger, agree on a verifier circuit C with one output wire, and C is garbled at setup. At dispute time the operator publishes its proof on-chain under one-time keys, and each published bit gives the evaluator the input label of that bit's value. The proof is public, so we use *privacy-free* garbling: the evaluator may learn every wire's value. What garbling must provide is *authenticity*. Holding the labels of an input x, the evaluator obtains the label of the output value $C(x)$, and it cannot obtain the label of the other value. A hashlock on the label of value 0 lets a challenger who evaluates an invalid proof stop the operator's claim.

The garbler may cheat, so the garbling is checked by cut-and-choose, as in BABE [GKS+26], Mosaic [KTA+26] and bitvm-gc [GOA26b]; its cost is linear in the size of the garbled circuit, which this paper reduces. Section 5 gives an integration sketch, accounted input-publication components and the unresolved deployment requirements.

3.2 The garbling scheme

We use privacy-free garbling [FNO15] with free-XOR [KS08] and one ciphertext per non-free gate, in the syntax of Bellare, Hoang and Rogaway [BHR12b].

Let Δ be a random secret 128-bit offset. Every wire w has a false label w_0, and its true label is $w_1 = w_0 \oplus \Delta$. Let $H(\ell, g) = \text{Blake3}(\ell \,\|\, g)$ truncated to 128 bits, where g is the gate's index. An XOR gate sets $c_0 = a_0 \oplus b_0$ and has no ciphertext. An AND gate $c = a \wedge b$ sets

$$c_0 = H(a_0, g), \qquad ct = H(a_1, g) \oplus H(a_0, g) \oplus b_0.$$

An evaluator holding the label a of value x, and b of value y, outputs $H(a, g)$ if $x = 0$ and $H(a, g) \oplus ct \oplus b$ if $x = 1$. In both cases this is the label of $x \wedge y$. An OR gate is symmetric, with $c_0 = H(a_1, g) \oplus \Delta$ and $ct = H(a_1, g) \oplus H(a_0, g) \oplus b_1$. The garbled circuit is one 16-byte ciphertext per AND or OR gate.

Two details matter in practice. First, Δ must be secret and fresh for each garbling: with a public Δ any label yields its complement, and the output's other label is free to compute. Second, a hash modeled as circular correlation robust (§7) gives labels no algebraic structure. RSA-based garbling proposals for BitVM3 were broken exactly through such structure: label forgery by extended Euclid on coprime exponents, and by the Franklin–Reiter related-message attack [FLB25].

3.3 The proof system

We target Plonky3's STARK verification pipeline [Plo26] over the Boolean WHIR trace commitment. The evaluated implementation instantiates one AIR, with no public values, preprocessed columns or lookups, and requires current- and next-row openings for every column. The pipeline uses fields in the Wiedemann tower [Wie88]

$$\mathsf{T}_0 = \mathbb{F}_2, \qquad \mathsf{T}_{k+1} = \mathsf{T}_k[X_k]/(X_k^2 + X_{k-1}X_k + 1), \qquad X_{-1} = 1,$$

up to $\mathsf{T}_7 = \mathbb{F}_{2^{128}}$. An element of T_{k+1} is stored as its low half (the coefficient of 1) and its high half (the coefficient of X_k), recursively down to bits. The trace is committed as bits: a Boolean column is packed into $\mathbb{F}_{2^{128}}$ elements, and WHIR commits to the packed multilinear polynomial on an additive (Cantor) evaluation domain [Can89, LCH14]. Binary towers and ring switching come from Binius [DP25, DP26].

Terms. An *AIR* (algebraic intermediate representation) is a set of polynomial constraints over a trace's current and next rows. A *zerocheck* [CBBZ23] proves that the constraints vanish on every row by a sumcheck [LFKN92] against the equality polynomial $\mathrm{eq}(\tau, \cdot)$ at a random point τ. *WHIR* [ACFY25] is a Reed–Solomon proximity test used as a multilinear polynomial commitment. Its *STIR queries* [ACFY24] open the committed, folded codeword at random positions, and its *out-of-domain* (OOD) samples bind the committed polynomial at points outside the evaluation domain. The *Cantor basis* is the $\mathbb{F}_2$-basis $v_0 = 1$, $v_i^2 + v_i = v_{i-1}$, which gives binary fields an additive FFT domain. The *bit ring switch* turns a claim about a polynomial's bits into a claim about its packed $\mathbb{F}_{2^{128}}$ representation. *Johnson list-decoding regime* refers to the proximity parameters for which the soundness bounds are proven via the Johnson bound, without conjectures.

Verification. The verifier:

1. runs a *zerocheck* of the AIR: a degree-4 sumcheck over the row variables, then the AIR constraints, evaluated at the bound point from the opened current and next rows and Horner-batched under a challenge α, against $\mathrm{eq}(\tau, r)$;

2. batches the opened columns into one bit-level claim, at a column point drawn from the transcript;

3. runs the *bit ring switch*, which turns that claim into an $\mathbb{F}_{2^{128}}$ claim on the packed polynomial. It uses a 128×128 tensor and, above 2^7 rows, its carry and last tensors, a batched sumcheck, and a closing weight computed in the tensor algebra;

4. runs the WHIR opening of that claim: sumcheck rounds with proof of work, STIR queries on Merkle-committed folded codewords, and a final polynomial.

All challenges come from a Blake3 [OANWO20] sponge over bytes, and all Merkle trees use Blake3 compressions.

4 The verifier as a circuit

This section turns the verifier of §3 into a Boolean circuit with one output wire. Figure 1 shows its stages in transcript order, with each stage's share of the non-free gates of our running example: a 2^{18}-row Keccak-f trace with 1,625 columns, 1,650 AIR constraints and 22 packed variables, whose proof is 140 KiB and whose verifier has 94.1M non-free gates. We first state the approach and the two choices that fix its cost, then describe the gadgets, the transcript, the zerocheck, batching and ring switch, and WHIR.

4.1 Approach

The construction has three parts. First, a deterministic circuit builder emits the STARK verifier of one configuration, an AIR, a trace height and a set of WHIR parameters, as a Boolean circuit over the proof's bits with one output wire. Every data-dependent branch of the verifier is fixed and guarded by a check, and the transcript follows the byte schedule recovered from the prover's own challenger, so the same circuit serves every proof of the configuration and is built before any proof exists. Second, gadgets for the two operations on which the verifier spends its gates, tower-field multiplication and Blake3 compression, under the free-XOR garbling of §3.2. Third, a streaming garbler runs the builder twice, once to plan each wire's lifetime and once to garble each gate as it is emitted, so the circuit itself is never stored (§6). At setup the operator garbles with every input at 0 and publishes the ciphertexts; at dispute the challenger obtains the input labels of the published proof and evaluates the same builder on them. Two choices fix the cost of the second part.

Why a binary field. The Keccak-f AIR is Boolean and its trace is committed as bits, so a prime-field pipeline would spend its arithmetic emulating bits, whereas in $\mathbb{F}_{2^{128}}$ addition is XOR, free under free-XOR, and the only AND gates are base-field products: 2,187 per multiplication by Karatsuba down the tower, with squaring and multiplication by the tower generator free. To price the prime-field alternative we also built KoalaBear ($p = 2^{31} - 2^{24} + 1$) and Poseidon2 on wires, to price the prime-field alternative. A KoalaBear multiplication is 3,451 AND gates, and a degree-4 extension multiplication 60,615, against 2,187 for all of $\mathbb{F}_{2^{128}}$. One Poseidon2 permutation of width 16 is 1,240,747 AND gates: 120 Blake3 compressions. The WHIR opening of a 2^{20} KoalaBear configuration (2,157 permutations, about 7,500 extension multiplications) would be about 3.1×10^9 gates with Poseidon2, more than the whole Boolean-garbled Groth16 verifier, and still $0.3\text{–}0.5 \times 10^9$ with Blake3, against our 21M for a 2^{18}-variable opening. The four-fold difference in size does not account for a gap of 15 to 150×: a garbled STARK verifier should use a binary field. The bit ring switch of Binius [DP26], which turns the bit-level claims into claims on the packed $\mathbb{F}_{2^{128}}$ polynomial, is the price of the choice: a third of the circuit (Figure 1).

Why Blake3. One hash serves three roles: the Merkle commitments and the Fiat–Shamir sponge of the proof system, and the garbling hash H of §3.2. Blake3 is an add-rotate-XOR design, so in Boolean gates its only non-free operations are the 32-bit additions, one AND per bit with the carry gadget of §4.2, which is the 120-fold gap to Poseidon2 above. A prover that already commits and

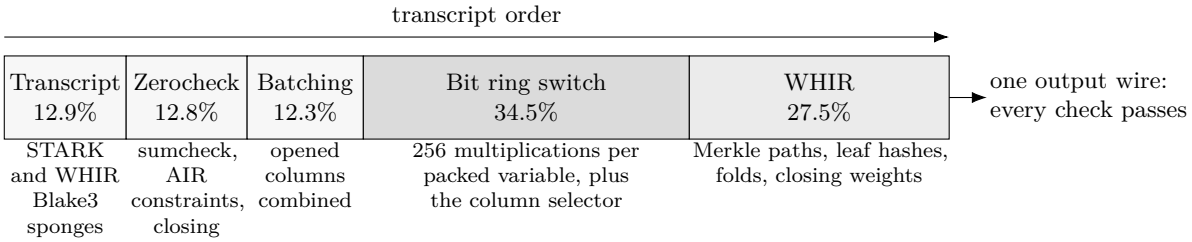


Figure 1: The ring switch dominates the verifier circuit, followed by WHIR. The stages appear in transcript order, and each segment's width is its share of the 94.1M non-free gates of the 2^{18}-row verifier: transcript is the STARK and WHIR sponges, zerocheck is its sumcheck, AIR constraints and closing, and WHIR is everything after the ring switch except its transcript.

samples with Blake3 needs no change, and the circuit's transcript can be checked byte for byte against it. A single hash also keeps the verifier's assumptions to those of §7, Blake3 as a random oracle with circular correlation robustness, with no elliptic-curve or lattice assumption, which is what makes the verifier's own security claim post-quantum (§7.1).

4.2 Gadgets

Tower arithmetic. A wire vector for an element of T_k is its bit pattern, so gadget outputs can be read back directly as field elements. Multiplication follows the tower recursively, Karatsuba at each level:

$$z_0 = a_0 b_0, \quad z_2 = a_1 b_1, \quad z_1 = (a_0 + a_1)(b_0 + b_1) + z_0 + z_2, \qquad ab = (z_0 + z_2) + (z_1 + X_{k-1} z_2)\, X_k.$$

Additions are free. Multiplication by the generator X_{k-1} is a rewiring plus XORs, and squaring is linear over $\mathbb{F}_2$. The only AND gates are the base-field products, three per level, so a 2^k-bit multiplication costs 3^k AND gates: 2,187 in $\mathbb{F}_{2^{128}}$. A multiplication by a constant folds to XORs entirely.

Blake3. We start from bitvm-gc's Blake3 circuit and change two things. A 32-bit addition's carry is the majority

$$\mathrm{maj}(a, b, c) = c \oplus \big((a \oplus c) \wedge (b \oplus c)\big),$$

one AND per bit, and the carry out of the top bit is discarded, so it is not computed. This halves a compression from 20,657 to 10,281 AND gates. We also add Blake3's tree mode (chunk chaining values on a stack, parent nodes, the root last). The verifier's transcript absorbs more than one 1,024-byte chunk between samples: the final polynomial is 64 coefficients, 1,024 bytes on its own. Table 1 gives the costs. Every gadget is checked against reference implementations of the field and the hash on random inputs.

4.3 Transcript

The transcript is a Blake3 byte sponge that mirrors the prover's challenger. Observing appends to an input buffer; sampling, when the output buffer is empty, hashes the input buffer, replaces it with the digest, and pops bytes from the digest's end. A field element is 16 little-endian bytes. A query index is 8 bytes read as a little-endian integer and masked. A proof-of-work check absorbs the witness and requires a fixed number of sampled bits to be zero. We implement the same challenger

gadget	AND gates
$\mathbb{F}_{2^{128}}$ multiplication	2,187
$\mathbb{F}_{2^{128}}$ squaring, multiplication by X_{k-1}, addition	0
Blake3 of 64 bytes (a Merkle compression)	10,281
Blake3 of 256 bytes (a leaf row of 16 elements)	41,511
Blake3 of 1,072 bytes (two chunks: a transcript flush)	186,125

Table 1: Gadget costs, checked against reference implementations of the field and the hash.

on wires. Its schedule has no data-dependent branch: queries are a fixed number of masked draws, and a proof-of-work check a fixed number of bits. So the circuit's shape is a function of the configuration alone. The sub-transcripts' domain separators are constants, baked in as constant wires, and everything the prover sends is an input.

The byte schedule, that is, the order in which roots, out-of-domain answers, sumcheck messages, proof-of-work witnesses, stratified query draws and the final polynomial are absorbed and sampled, is implicit in the prover's implementation and specified nowhere else. We recover it by running that implementation's own verifier through a challenger that logs every byte, and we test the circuit against the log op for op.

4.4 Zerocheck, batching and ring switch

The zerocheck, the column batching and the ring switch are implemented gate for gate from a reference implementation that is checked op for op against the prover's verifier. The AIR arrives as symbolic constraints and is evaluated on wires.

Two data-dependent branches of the reference verifier cannot appear in a fixed circuit. We fix them and guard them with checks.

- The verifier redraws a zerocheck challenge coordinate that happens to be zero. The circuit takes a fixed number of draws and requires each to be nonzero.
- The ring switch would fix a column-point coordinate that is exactly 0 or 1. The circuit assumes there is no such coordinate and checks it.

Each event has probability about 2^{-127} per coordinate. When it occurs the circuit rejects where the reference verifier would continue, never the reverse.

4.5 WHIR

The WHIR layer checks every proof-of-work witness. It hashes every opened row and walks it up its Merkle path to the root absorbed in the transcript. It then folds each queried row with the round's folding randomness, and checks the final queries against the final polynomial and the closing sumcheck identity. The prover sends Merkle openings pruned: siblings that another query can recompute are left out. Our implementation expands the proof outside the circuit into one full path per query. This is an engineering choice rather than an inherent limitation of a fixed circuit: a fixed upper-bound circuit could verify a padded frontier with data-dependent routing. A million-trial simulation of the actual stratified queries and cap forest, however, reduces the expected path from 1,354 to 1,335.71 digests, only 4,683 input bits or 0.45% of the total, before the routing and padding overhead. When the commitment uses a Merkle cap of 2^h roots, the transcript absorbs all of them, and each query selects its root with a multiplexer on the top h index bits: $256(2^h - 1)$ AND gates, against h compressions saved.

Implementation. We instantiate the construction on Plonky3 [Plo26], whose Blake3 challenger, symbolic AIR constraints and Boolean WHIR commitment are the ones described in §3. The gadgets are checked against its binary-field crate and the `blake3` crate, the byte schedule is logged from its verifier, and the circuit is tested against that log; §6 reports the measurements.

5 Bitcoin integration

This section places the garbled verifier in a BitVM3-style dispute [WAA+26, KTA+26], in which the operator garbles and challengers evaluate, with a bridge as the motivating application. The roles follow from the construction: evaluation is one Blake3 call per non-free gate and needs no secret, so any party holding the kept garblings can check a claim and stop a fraud, and the garbler's cost is paid once at setup. The reverse assignment would also make the operator's payment depend on a challenger handing over its input labels. It fixes the setup and the intended dispute, states the label-selection task that any way of publishing the proof must solve, compares the adaptor-signature and hash-based solutions on the same terms, and prices four one-time input schemes that we serialized as complete signed transactions and replayed through Bitcoin Core's strict standard policy. Appendix A holds the per-scheme fixtures and a post-quantum instantiation.

The accounting boundary is as follows. We use *total on-chain footprint* to mean the sum of the serialized virtual sizes, in virtual bytes (vB), of every transaction in the dispute graph, including ordinary transaction signatures, authentication signatures or preimages, witness framing, revealed tapscripts, control blocks, transaction inputs and outputs, anchors, timeouts and fee-management transactions. We measure the input-publication component of that footprint: bounded funding–reveal publication slices for all four schemes, and a signed join for Lamport. The challenge, timeout, Disprove, anchor and fee-management branches, the hash-based soldering across the kept garblings and the binding of public values are not constructed, so the figures are measured components and conditional projections, not a total on-chain cost.

Setup. The operator garbles N instances of C from independent seeds and commits, for every instance j, the hash of its garbled tables, $h_j = \mathrm{SHA256}(\ell_0^j)$ for the label of output value 0 (reject), and the hashes of its one-time keys and label translations. A challenger chooses $N - M$ instances to open, re-garbles each from its seed in one pass of the plan that all instances share (§6) and checks every commitment; only then is the transaction graph signed, by a committee as in BitVM2 [LAA+25], whose enforcement needs one honest signer. At $(N, M) = (181, 7)$ and 2^{18} rows the operator garbles 181 instances at about 57 s each, 2.9 core-hours or about 11 minutes on 16 cores, and a challenger stores the 7 kept instances, 1.5 GB each or 10.5 GB; BitVM3 reports 3 hours 24 minutes on 16 cores and 288.4 GB for the Boolean-garbled Groth16 verifier at the same parameters [WAA+26]. These setup figures are derived from the measured garbling time, not measured end to end. Publishing bit b of input i must give a challenger only the label of value b on wire i in every kept instance; Mosaic provides this correlation for its adaptor-signature path [KTA+26], and the hash-based soldering is open here.

Figure 2 summarizes the intended logical outcomes for one claim. The Assert node is a placeholder for an unconstructed transaction subgraph.

Required transaction semantics. The intended branches are:

- *Kickoff*: the operator claims reimbursement for a withdrawal it has paid. If no challenge follows within Δ_C, *Take1* pays it. This is the happy path.

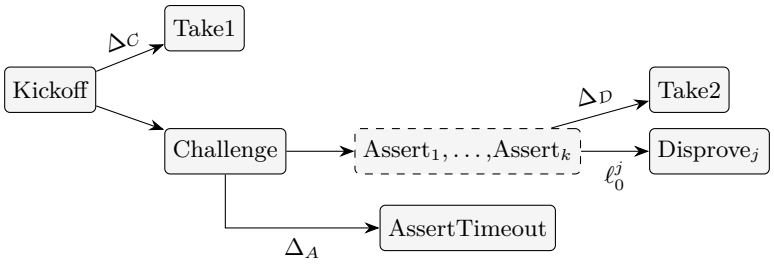


Figure 2: Logical outcomes of the proposed dispute. The dashed Assert$_1$, . . . ,Assert$_k$ node denotes an unspecified publication subgraph. The figure does not define its UTXO dependencies, completion anchor, timeout enforcement or fee management, all of which remain to be constructed and policy-tested.

- *Challenge*: any challenger may spend Kickoff's challenge output, paying the operator a bond that covers the cost of Assert, as BitVM2's crowdfunded challenge does [LAA+25].
- *Assert*$_1$, . . . ,*Assert*$_k$: the operator must publish every authenticated circuit-input bit. Because the payload exceeds one standard transaction, a concrete construction needs a subgraph whose terminal output is spendable only after every chunk has been published. We do not specify that fanout, chaining or completion mechanism here.
- *AssertTimeout*: if the operator fails to create the completion output before Δ_A, a timeout branch should transfer its stake. The enforceable condition for "all chunks published" depends on the missing subgraph and is not specified by Figure 2.
- *Disprove*$_j$: after such a completion output exists, a challenger holding ℓ_0^j would open the corresponding hashlock before Δ_D; otherwise Take2 would pay the operator. The required transaction dependencies and relative timelocks remain unimplemented.

The complete graph must be tested against the release policy targeted for deployment; current policy limits a standard transaction to 400,000 weight units and an unconfirmed cluster to 101 virtual kilobytes [Bit26].

Intended dispute. A challenger reads the published bits, derives the input labels of every kept instance and evaluates each off-chain, one Blake3 call per non-free gate, 17 s at 2^8 rows (§6). An invalid proof yields ℓ_0^j from a correctly garbled instance, which opens Disprove$_j$. The circuit size affects off-chain evaluation, while the proof size controls the Assert payload. The four authenticated one-key-set rows of Table 3 serialize complete signed reveal transactions under BIP341's nothing-up-my-sleeve (NUMS) internal key, execute their leaves with real sighash contexts and pass Bitcoin Core's strict standard-policy mempool after funding confirmation; only Lamport includes a signed join. The evaluated circuit has no public values, so binding the withdrawal and rollup-state commitment into the AIR relation and transcript, which a bridge needs, is unimplemented and outside the reported figures.

Security. Let ε_{cc} bound the probability that every kept instance is malformed while every opened one is correct; for choosing $N - M$ of N uniformly this is $1/\binom{N}{M}$, which is 2^{-40} at $(N, M) = (181, 7)$ [GKS+26]. Let ε_{sol} bound a failure of the soldering, and let ε_{sig} bound the aggregate probability of forging any one-time or transaction-authorization signature across the entire dispute,

so a reduction from a per-target bound must include the number of targets. At these parameters, cut-and-choose alone limits the intended malicious-garbler security to about 40 bits, irrespective of the STARK verifier's 104-bit classical soundness. Subject to an enforced transaction graph, timely confirmation and an instantiated soldering bound, Theorem 1 of §7 supports two intended properties. *A false claim is disproved:* if the operator's statement is false, an honest challenger obtains some ℓ_0^j except with probability $\varepsilon_{\mathrm{STARK}}(t) + \varepsilon_{\mathrm{cc}} + \varepsilon_{\mathrm{sol}}$, since a correctly garbled kept instance exists, its labels are those of the published proof, and Theorem 1(i) applies; an operator that publishes no proof loses its stake to AssertTimeout. *An honest operator keeps its stake:* if the proof is honest, no challenger obtains any ℓ_0^j except with probability $M(\varepsilon_{\mathrm{guard}} + \varepsilon_{\mathrm{GC}}) + \varepsilon_{\mathrm{sol}} + \varepsilon_{\mathrm{sig}}$, by Theorem 1(ii) and a union bound over the kept instances. This is conditional accounting, not a protocol theorem in a ledger model: $\varepsilon_{\mathrm{sol}}$ has no instantiated bound, and the transaction graph and ledger-timing assumptions have not been implemented or proved. BitVM3 [WAA+26] and BABE [GKS+26] analyze their own instantiations.

Label selection. Every input mechanism we price is an instance of one two-party task. The *garbler* G (the operator, who also proves) holds the two labels ℓ_w^0, ℓ_w^1 of every input wire w of the kept garblings; the *evaluator* E (a challenger) will run the garbled verifier on the proof π. Publishing π on-chain must (i) *deliver* $\ell_w^{\pi_w}$ to E for every w, (ii) be *exclusive*: no party learns any $\ell_w^{1-\pi_w}$, since one complementary label breaks authenticity (§7), (iii) be *binding*: G publishes at most one value per wire, and the publication is also the authorization of the spending transaction, and (iv) be *public*: any observer can recover π and check the published object. A malformed publication cannot be spent, a missing one forfeits the stake through AssertTimeout, and Disprove is reserved for the reject label. Lamport achieves (i)–(iv) with one preimage per bit, at the cost of a per-bit script. Figure 3, the adaptor construction of BitVM3 and Mosaic [WAA+26, KTA+26], achieves them with one signature per k-bit digit and no per-bit script. The hash-based alternative (Figure 5, Appendix A.1) replaces the signature by a pair of hash-chain openings and underlies the Antichain row and the hash-based projections of §7.1.

Both protocols publish the same selection material per digit, one 64-byte completion or two n-byte nodes: at $k = 8$ for the adaptor and $k = 4$, $n = 16$ for the hash-based fixture, about 8.1 bytes of witness per bit, and the 130,128 adaptor completions and 16,266 hash records of our fixtures are both exactly 8,458,320 bytes. They differ in where the remaining cost sits. The adaptor's selection is verified by one `CHECKSIG` per digit, 3 bytes of script, and its binding is the same signature; its setup holds 2^k secrets and ciphertext bundles per digit, and its exclusivity rests on discrete logarithms, which Shor's algorithm removes (§7.1). The hash-based selection has a two-node setup and hash-only exclusivity, but under current consensus every hash step of the verification is 2 bytes of script, plus a separate transaction signature; that script makes the Antichain row 5.3 times the adaptor row in Table 3. The digit-to-label translation table is the same object in both and is not on-chain. With a native batch opcode, a hash-based transaction signature and full-width nodes, the hash-based selection would match the adaptor's 2.13 MvB at about 117 quantum bits; we serialized that construction exactly, but none of its three ingredients exists on Bitcoin today (Table 7, Appendix A.2).

Publishing the input. We price four one-time schemes for carrying the input bits; Appendix A.3 gives their scripts and fixtures and Table 3 their sizes. *Lamport* reveals one of two hashed 16-byte preimages per bit against a 49-byte leaf script, 998 bits per input, with a 32-byte authorization key and `OP_CHECKSIGVERIFY` prefixed to each leaf. *Antichain Winternitz* [TF26a] closes the forward-hashing gap of plain Winternitz chains at a cited marginal 11.31 vB per bit at parameters $(2, 16)$;

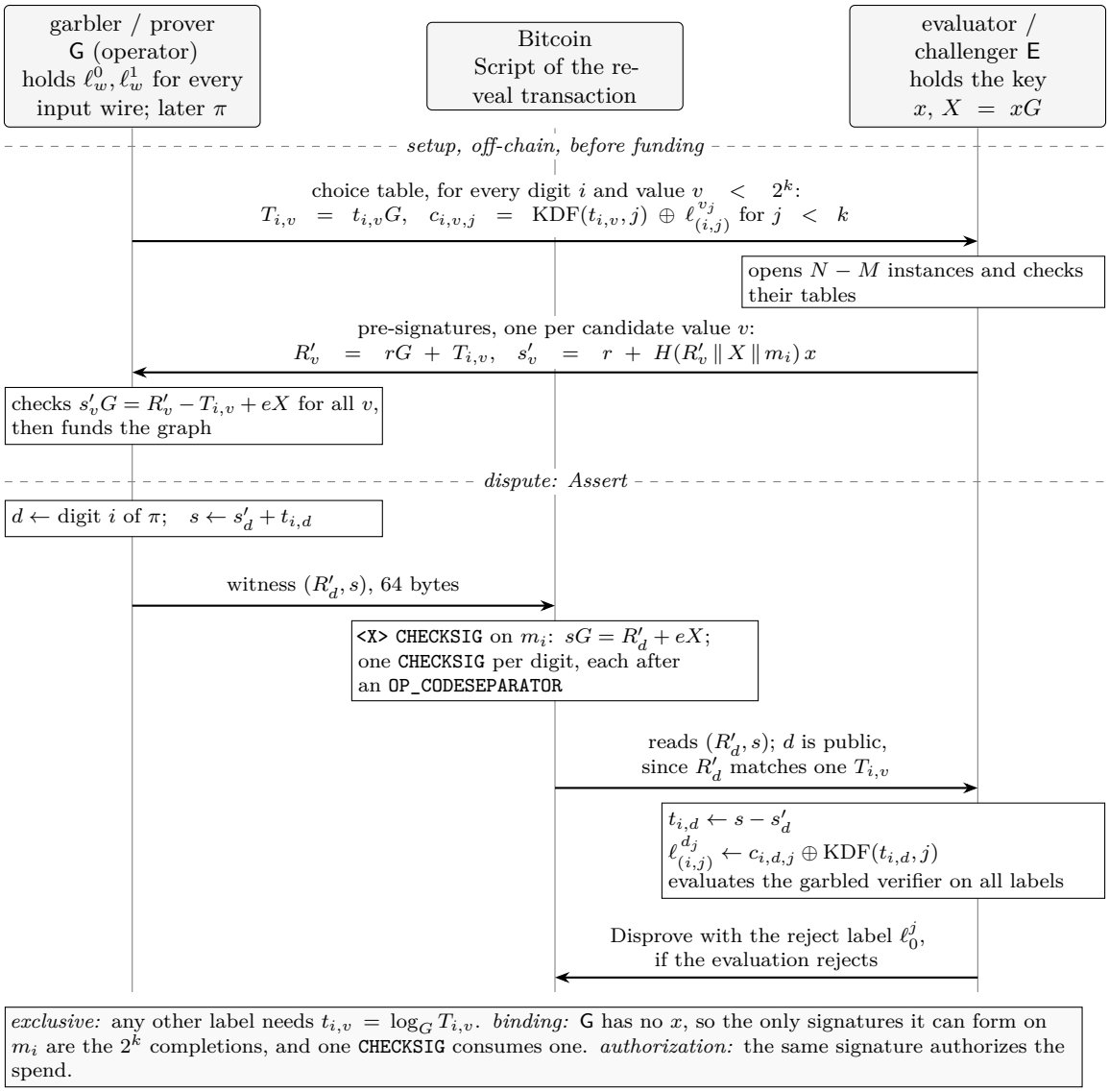


Figure 3: Schnorr adaptor label selection for one k-bit digit position i, as in BitVM3 and Mosaic [WAA+26, KTA+26]. m_i is the BIP341 message seen by the i-th CHECKSIG; wire (i, j) is bit j of the digit. Everything above the dispute line is off-chain and repeated for every digit and every candidate value.

our fixture pins the upstream implementation and adds a 64-byte Schnorr authorization. *Winternitz with a translation table* [SST+26] closes the same gap by making each label a Shamir secret whose threshold a signed chunk's checksum sets; our fixture uses 3-bit digits, eight per chunk, at the price of a 1.905 GB off-chain table per instance. *Schnorr adaptor signatures* over 8-bit digits, as in BitVM3 and Mosaic, reproduce BitVM3's 9,323 weight units per 1,024-bit Assert shell and validate every completed signature under Core 31.1; the signatures rest on discrete logarithms, and this repository does not implement Mosaic's polynomial soldering. Packing more bits into each adaptor completion shrinks the serialized transactions to 1.11 MvB at 16 bits, but the off-chain choice table grows to 277 GB per key set and the exclusive delivery of binary labels for such digits is unimplemented, so the 8-bit construction remains the authenticated baseline.

Our labels are free-XOR correlated, so a hash-based preimage s cannot itself be a label: a key-derivation function (KDF) maps it to one by a translation ciphertext $\mathrm{KDF}(s) \oplus w_b$, two per input bit and instance, 33 MB per instance at 2^{18} rows. Using one on-chain key set for all M kept instances requires a proof that the translations of opened and kept instances encode the same bit choices. BABE's zero-knowledge soldering [GKS+26] and Glock's verifiable secret sharing [Eag25] give related mechanisms at smaller input sizes; we have neither constructed nor costed one for a million bits, so the one-key-set hash rows of Table 3 are conditional on it and $\varepsilon_{\mathrm{sol}}$ is unbounded. Authenticating each input independently for every kept instance instead multiplies the component by M: at $M = 7$, 120.86 MvB for Lamport, 82.94 MvB for Antichain and 55.03 MvB for WOTS translation, before graph overhead and still without enforcing the same bit choice across the sets.

We also ran the dispute components, with the on-chain steps executed as Bitcoin Script, on a one-compression Blake3 preimage statement and on the Keccak-f verifier at 2^5 rows. For one garbling, the Assert publication, label translation, evaluation and reject-label hashlock behaved as intended (Appendix A.4); signatures, timelocks and cut-and-choose were not part of either run.

6 Evaluation

The experiments answer five questions: how the garbled verifier scales with the trace (Table 2, Figure 4), where its gates go and what each design choice contributes, how the circuit-input authentication component projects (Table 3), how far the input shrinks, and how the result compares with estimated STARK verifiers and pairing-based designs (Table 4). Appendix B holds the gate profile, the ablation, the Core replay, the input sweep and the recursion details.

Setup. We proved Keccak-f permutations with Plonky3's 1,625-column characteristic-2 Keccak AIR, at rate 1/32, folding factor 4 and 110 bits per WHIR term; booleanity is not constrained, since the commitment is to bits. A byte-logging challenger recorded each run, our reference replayed Plonky3's verifier byte for byte, and the verifier circuit was then streamed through the single-threaded garbler. Runs up to 2^{18} rows had 32 GB of RAM and eight cores of an Intel Xeon Platinum 8375C at 2.9 GHz; the 2^{20} run used an AMD EPYC 9355 limited to 128 GB and 32 cores. Appendix B.1 lists the pinned revisions and commands. Gate, wire and input counts are deterministic; wall-clock times are single-run measurements unless a range is stated. The circuit's size depends on the AIR, since the transcript, column combination and constraint evaluation grow with the columns and constraints; all sizes are for this 1,625-column AIR, a demanding case, whereas a Groth16 verifier's circuit is fixed for a given verifying key and public-input arity.

Scaling. Table 2 and Figure 4 give the measured verifier configuration from 2^5 to 2^{20} rows. Every circuit accepted its proof, and at 2^5 and 2^8 we also checked that it rejected the proof with one opened value changed. Growing the trace 32,768-fold grew the circuit 2.2-fold, because the verifier depends on the trace length only through the logarithmic number of variables.

Garbling efficiency. To compare garbled verifiers across trace sizes and AIRs, we define the *garbling efficiency* of a configuration as

$$\eta = \frac{|\mathsf{GC}|}{R \cdot W} \quad \text{bytes per committed cell,}$$

where $|\mathsf{GC}|$ is the garbled circuit's size in bytes, 16 per non-free gate, R the number of trace rows and W the number of committed columns. It is a cost per unit of verified computation, so smaller

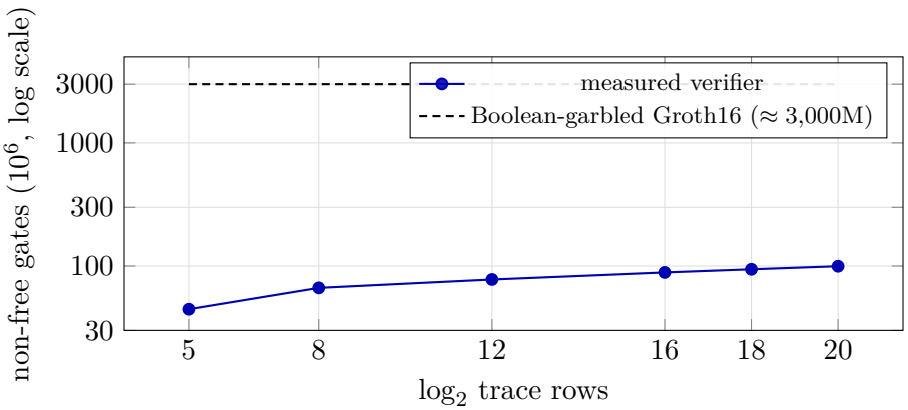


Figure 4: On a logarithmic vertical axis, the measured verifier grows from 44.6M non-free gates at 2^5 rows to 99.6M at 2^{20}, a 2.2× increase for a 32,768× larger trace. The dashed line shows the approximately 3,000M-gate Boolean-garbled Groth16 comparator.

trace	vars	non-free gates	garbled	η (B/cell)	garbling	garbler memory	input bits
2^5 rows (1 Keccak-f)	9	44,573,121	713 MB	13,715	35 s	0.32 GiB	586,240
2^8 rows (10)	12	66,459,859	1,063 MB	2,556	53 s	0.46 GiB	743,808
2^{12} rows (163)	16	77,736,394	1,243 MB	186.9	53 s	0.54 GiB	874,240
2^{16} rows (2,621)	20	88,680,747	1,418 MB	13.32	66 s	0.61 GiB	997,760
2^{18} rows (10,485)	22	**94,069,548**	**1,505 MB**	**3.53**	57 s	0.65 GiB	1,041,024
2^{20} rows (41,943)	24	99,613,898	1,593 MB	0.94	42 s†	0.68 GiB	1,108,992

Table 2: The measured verifier configuration stays below 100M non-free gates and 1.6 GB garbled up to 2^{20} rows, and the garbler needs less than 0.7 GiB. Measured on real Keccak-f proofs; "vars" is the number of packed variables WHIR opens, garbled size is 16 bytes per non-free gate, and η is the garbled size per committed trace cell, 1,625 columns times the rows. Garbling is single-threaded, and its time varied by up to 50% with load on the shared machine (23–35 s at 2^5 across runs). Garbler memory is the plan (one byte per wire) plus 18 bytes per live wire at the peak, computed from the measured counts; at 2^{18} rows the garbler holds at most 2,256,536 labels live. †The 2^{20} proof was generated on a second, larger machine, where the circuit was also garbled, again on one core.

is better. For our Keccak-f AIR, $W = 1{,}625$ and each cell is one bit. Because $|\mathsf{GC}|$ grows with $\log R$ while $R \cdot W$ grows with R, η falls almost in inverse proportion to the trace: from 13,715 bytes per cell at 2^5 rows to 3.53 at 2^{18} and 0.94 at 2^{20} (Table 2). At 2^{20} rows this is less than one garbled byte per committed bit: 7.5 garbled bits per bit, or 38 kB per Keccak-f permutation. Comparing η across proof systems needs the same computation and cells of the same kind.

Where the gates go. Table 8 in Appendix B.2 breaks the 2^{18} circuit down by the verifier phase that emits each gate. The stages before WHIR account for 70.8% and WHIR for 29.2%. The largest single item is the ring switch closing, 33.6%: 256 multiplications per packed variable, plus a fixed 11×512 for the column selector. The STARK transcript, the column combination and the AIR constraints, 34.9% together, grow with the number of columns, so a narrower AIR shrinks them directly. On a standalone instance, the WHIR opening of an 18-variable polynomial costs 21.2M non-free gates with a Merkle cap of 32 roots, three quarters of it hashing, and rate 1/32 with folding 4 was the smallest configuration in our sweep (Appendix B.2).

Streaming garbler. A builder that stores the gate list, as bitvm-gc's `CircuitAdapter` does, needs tens of gigabytes for these circuits before garbling starts, yet a gate's ciphertext depends only on its input labels. Our garbler is a builder backend that runs the deterministic builder twice: a plan pass counts each wire's reads, one byte per wire, and the garbling pass garbles each gate as it is emitted, giving wires slots that are released after their last read, with the constants and outputs pinned. At 2^{18} rows the plan is 0.61 GiB for 659M wires and the live wires peak at 2.26M, 41 MB at 18 bytes each; the plan depends only on the circuit, so every garbled instance of a configuration shares it. When the garbler holds a witness it also evaluates each gate and checks the result; at setup it garbles with every input at 0, and a challenger later evaluates the stored ciphertexts with a backend that holds one label per wire. Tested this way at 2^5 and 2^8 rows, the stored garbling gave the accept label on a real proof and the reject label on the proof with one opened value changed; evaluation took 11.5 s and 17.0 s against 20.6 s and 34.0 s for the garbling pass alone, since evaluation hashes once per non-free gate where garbling hashes twice.

Ablation. Replacing our Blake3 adder with the original two-AND adder, with everything else unchanged, grew the circuit by 32% at 2^5 rows and 29% at 2^8: the adder saves a quarter of the whole verifier, because Blake3 is in the transcript, the leaf hashes and every Merkle path. Building the same 2^5-row verifier as a stored circuit produced exactly the same gates and wires, but took 12.4 GiB and 283 s, against 0.69 GiB and 33 s for the plan and the streamed garbling together (Table 9, Appendix B.2).

Accounted input-publication components. A Taproot output commits to its tree through a 32-byte output key at funding time; the large cost occurs if the dispute branch is spent and the witness reveals the authentication signature or preimage, leaf script and control block. Table 3 accounts for that dispute-time component for the 2^{18}-row circuit, which reads 1,041,024 logical bits (127.1 KiB) from a 140 KiB proof. Every authenticated row includes its signature or opening witness, transaction-binding authorization, authentication script and complete reveal-transaction shell, with two P2TR outputs per transaction for the adaptor, WOTS-translation and Antichain fixtures and one for Lamport; the table compares concrete components, not complete or identical dispute graphs. Two transport rows are reference points, not lower bounds: a raw-data fixture of 1,627 80-byte witness items over two one-leaf P2TR inputs, 33,299 vB with no signature or label authentication, and a Core-accepted signed envelope in the style of ESSPI, BitVMX's signed program-input transport [LJF25], 33,004 vB, which authenticates the bytes but delivers no selected labels.

For one key set, the executable fixtures measure 2.22 MvB with the 8-bit Schnorr-adaptor baseline, 7.86 MvB with translated Winternitz, 11.85 MvB with safe Antichain Winternitz and 17.27 MvB with Lamport; the two hash-based figures are all-nonzero worst cases, and the adaptor's largest transaction is 64 WU below the 400,000-WU standard limit. All selected leaves execute with real BIP341 contexts under Bitcoin Core 31.1 strict policy. On isolated regtest nodes with non-standard transactions disabled, Core rejected each reveal as `too-large-cluster` while its funding parent was unconfirmed, accepted every reveal after confirmation, mining the four fixtures in 3, 9, 12 and 18 blocks, and accepted Lamport's 13,060-vB signed join only after its 174 reveals confirmed (Appendix B.3). This establishes staged standard-policy feasibility for bounded signed publication slices, not propagation, fee management, timeout trees or the missing challenge, Disprove, anchor and soldering graph. At BABE's historical 2.2 sat/vB and $95,500 per bitcoin [GKS+26], the four components correspond to 0.049, 0.173, 0.261 and 0.380 BTC, neither current quotes nor fees for a complete dispute. The one-set hash rows require the unimplemented soldering of §5, and the

input mechanism	key sets	vB/bit	accounted component	evidence and accounting boundary
raw transport baseline	0	0.032	33.30 kvB	serialized two-input fixture: data, drop scripts, control blocks and tx shell; no signature or label authentication
signed envelope transport [LJF25]	0	0.032	33.00 kvB	Core-tested commit and signed reveal; authenticates bytes but supplies no selected GC labels and excludes the data-availability and fraud graph
Schnorr adaptor, 8-bit baseline [WAA+26, KTA+26]	1	2.13	2.22 MvB	Core-tested 133-input, 23-transaction fixture; completed signatures, code separators, scripts, depth-zero controls and two P2TR outputs
WOTS translation [SST+26]	1	7.55	7.86 MvB	Core-tested all-nonzero worst case: 886 signed inputs in 81 two-output transactions; off-chain table 1.9 GB per instance
Antichain $(2, 16)$ [TF26a]	1	11.38	11.85 MvB	Core-tested all-nonzero worst case: 784 signed inputs in 131 two-output transactions using pinned upstream openings and scripts
Lamport	1	16.59	17.27 MvB	fully serialized 174-reveal fixture: transaction-binding signatures, preimages, scripts, depth-zero controls, input/output shells; Core 31.1 accepted after funding confirmation
WOTS translation, independent keys	7	52.86	55.03 MvB	arithmetic 7× worst-case projection; consistency and graph unimplemented
Antichain, independent keys	7	79.67	82.94 MvB	arithmetic 7× worst-case projection; consistency and graph unimplemented
Lamport, independent keys	7	116.10	120.86 MvB	arithmetic 7× projection; consistency and graph unimplemented

Table 3: Source-specific accounting of the dispute-time circuit-input publication component for 1,041,024 logical bits. The adaptor row includes completed transaction-binding Schnorr signatures. Each hash-based row includes its one-time opening witness plus an ordinary transaction-binding Schnorr signature. Every label-authentication row also includes the authentication scripts, control blocks, input/output shells and reveal-transaction framing. Adaptor, WOTS translation and Antichain use two outputs per transaction, whereas Lamport uses one. Funding for all four and Lamport completion are reported separately in the text. The complete challenge/timeout/Disprove graph, anchors, robust fee management and multi-copy soldering are not included. The table therefore does not report total on-chain cost.

seven-key-set rows are its arithmetic projection, which proves no consistency of bits across the sets.

How far the input shrinks. We counted the verifier's input bits by proof component from the WHIR schedule alone; the count equals the measured input at the five tested configurations through 2^{18} rows. Sweeping rates 1/8 to 1/256, folding factors 2 to 6 and grinding budgets up to 48 bits at 2^{18} rows, keeping 110 bits per WHIR term in the Johnson regime, the smallest input, at rate 1/256, folding 4 and 48 bits of grinding, is 845,952 bits, 19% below the measured configuration, at 103.9 bits of composed soundness (Table 10, Appendix B.4). Its WHIR part shrinks by 35%, but 480,256 bits do not depend on WHIR at all: the opened values are 416,000 bits, two rows of 1,625 columns at 128 bits each. The tuning also moves soundness from queries to grinding, which halves under quantum search (§7.1). Below this the input shrinks only with a narrower AIR or by recursion. Narrowing the AIR while keeping at least the measured $1{,}625 \times 2^{18}$ committed cells, hypothetical widths 128 and 32 at heights 2^{22} and 2^{24} need 659,840 and 636,288 input bits under the current WHIR profile, and 464,768 and 441,216 bits with the best swept configuration, reductions of 37–58%; these are exact schedule counts for layout models, not generated proofs, since no such narrow Keccak AIR exists. Batching has real artifacts at both ends: the 2^{18} and 2^{20} traces hold 10,485 and 41,943 permutations for 1,041,024 and 1,108,992 input bits, so four times the work adds 6.53% to the input and amortizes it from 99.29 to 26.44 bits per permutation, 3.76×, without reducing

the absolute unhappy-path payload of one proof.

Recursion into a narrow machine. The floor above belongs to the statement, so we also garbled the verifier of a statement built to be verified cheaply. The binary stage of the MIPS32 zkVM Ziren [DYH+26, ZKM26] proves, over $\mathbb{F}_{2^{128}}$ with Boolean WHIR and Blake3, the program that verifies a shrunk zkVM proof, and records Plonky3's verifier of a binary-stage proof as a straight-line tape over $\mathbb{F}_{2^{128}}$; a five-table tape machine proves a run of the tape, and its own verifier is recorded and proved again. We translate a tape gate by gate, trusting Ziren's recorder. For a Fibonacci program the binary stage's verifier has 3.37G non-free gates and 18,050,304 input bits; after three rounds of narrow recursion, with the recording choices and five unreviewed proof-system changes of Appendix B.5, the verifier of the last proof has 564,552 input bits, made of 9,513 bytes and 3,816 field elements, 0.54× the Keccak verifier's, in 218.0M non-free gates and 3.49 GB (Table 11). Every row of that table is the verifier of a real proof, garbled by the streaming garbler; each accepts its proof and rejects three single-input changes. The runs bind no public values, so the circuit attests that some zkVM proof verifies, not which program ran; binding the public-value digest would add at most 1,024 input bits. Each proof meets 100 bits of composed soundness by Plonky3's assessment, with up to 46 bits of grinding per term, which halves under quantum search. At 564,552 bits, the fixtures of Table 3 serialize to 1.21 MvB for the adaptor baseline, 4.26 MvB for WOTS translation, 6.43 MvB for Antichain and 9.36 MvB for Lamport, not replayed in Core. Tiwari and Feickert [TF26b] estimated the Boolean verifiers of three zkVMs' recursion proofs from their operation counts (Table 12); the smallest, RISC Zero's with Blake3 in place of its internal hash, has 1.56G non-free gates, 7.2× our measured recursion verifier, whose input is 3.2× smaller than that proof.

Comparison. Table 4 places the result among approaches to verifying proofs on Bitcoin (§2). For the designated-verifier pairing SNARK we garbled bitvm-gc's circuit with our streaming garbler: 481.6M non-free gates and 1.92×10^9 wires, at most 15.6M live, 2.2 GB peak memory in under six minutes where a stored build needed more than 64 GB; it accepted the valid proof and rejected it with one witness bit flipped. The Keccak verifier has 5.1× fewer non-free gates, and 30× fewer than the Boolean-garbled Groth16 verifier; the recursion verifier has 2.2× and about 14× fewer. Native-group garbling of Groth16 [KFLT26] and witness encryption [GKS+26] produce artifacts of megabytes and need far fewer input bits, on pairings and a trusted setup. On-chain, the comparison inverts: the Groth16 designs publish 508 to 1,019 input bits and report dispute paths of a few to a few tens of kilobytes, under $60, whereas our Keccak dispute publishes 1,041,024 bits at a Core-tested input component of 2.22 MvB with adaptors or 7.86 MvB hash-based, and the recursion verifier's 564,552 bits serialize to 1.21 and 4.26 MvB. That is one to three orders of magnitude more, about 38× deferred binding's 32 kvB even for the recursion verifier with adaptors, before the unconstructed remainder of the graph: the price of verifying the STARK directly rather than wrapping it, which §5 and §8 are about reducing.

approach	statement	off-chain size	input bits	on-chain dispute path, as reported	assumptions
Boolean garbled verifier [Bit25]	Groth16	42 GiB	1,019	Assert 2.4 kvB and Disprove 93 vB, about $5 at 2 sat/vB [WAA+26]; the published experiment normalized to $37.65 [GKS+26]	pairings, trusted setup
native-group garbling [KFLT26]	Groth16	2.4 MiB	n/r	n/r	pairings, trusted setup
garbled DV-Pari (bitvm-gc, our measurement)	DV-SNARK	7.7 GB	n/r	n/r	elliptic curves, secret trapdoor
BABE [GKS+26]	Groth16	22.2 MiB	508	Assert 9,240 vB, ChallengeAssert 17,400 vB, WronglyChallenged 149 vB: 26.8 kvB, $56.90 at 2.2 sat/vB	pairings (generic group and random-oracle models), trusted setup
deferred-binding WE [Pha26]	Groth16, late inputs	0.92 GiB†	768	Assert 2,154 B, ChallengeAssert 14,312 B witness and 109,393 B script, WronglyChallenged 64 B; about 32 kvB	BABE + DLog
PIPE with AADP WE [ACG+26, SRG+26]	pairing SNARK	≈ 338 TB (estimate)	one hash	one hash in Assert, one signature in Disprove; no size reported	heuristic
this work	**Keccak-f STARK over $\mathbb{F}_{2^{128}}$, 2^{18} rows**	**1.5 GB**	**1,041,024**	**input publication 2.22 MvB (adaptor, DLog) or 7.86 MvB (hash-based), about $4,700 or $16,500 at 2.2 sat/vB, Core-replayed; Disprove 97 vB; graph incomplete§**	**Blake3 as a random oracle; assumed circular correlation robustness and adaptive authenticity‡**
this work, narrow recursion	STARK recursion proof over $\mathbb{F}_{2^{128}}$	3.49 GB	564,552	input publication 1.21 MvB (adaptor, DLog) or 4.26 MvB (hash-based), about $2,500 or $9,000 at 2.2 sat/vB; Disprove 97 vB; graph incomplete§	as above, plus unreviewed proof-system changes‡

Table 4: Approaches to verifying proofs on Bitcoin: off-chain artifact per instance (garbled circuits at 16 bytes per non-free gate), input bits published at dispute time, and on-chain dispute path, each as its source reports it; n/r: not reported. †One instance: 59.3M non-free gates in three garbled circuits plus adaptor tables, sharing a 3.83 GiB circuit artifact; the kvB total is our arithmetic. §One key set; the hash-based figure is WOTS translation, whose table adds 1.9 GB (Keccak) or 1.0 GB (recursion) per instance. Both rows serialize the reveal fixtures of Table 3 and add the reject-label Disprove; the rest of the graph is not constructed (§5), so each is a lower bound. ‡Verifier assumptions only; the protocol adds discrete-logarithm signatures, or the unimplemented hash-based soldering and post-quantum authorization. Neither measured run binds public values (§6). The recursion row, the last proof of Table 11, uses the unreviewed changes of Appendix B.5, and its statement, a Ziren proof, rests on an elliptic-curve digest [DYH+26].

7 Security

This section states what the garbled verifier guarantees and on which assumptions. It combines the soundness of the proof system, the effect of the two guarded branches, and the authenticity of the garbling.

The circuit and the verifier. When neither guarded event of §4 occurs, the circuit computes exactly Plonky3's verifier; we test this op for op against Plonky3's logged transcript and on real

trace rows	2^5	2^8	2^{12}	2^{16}	2^{18}	2^{20}
composed soundness (bits)	106.1	105.2	104.7	104.3	104.3	104.0

Table 5: Soundness stays at about 104 bits at every measured size, 103.95 at 2^{20} rows. The values are Plonky3's composed soundness for each configuration.

proofs. When a guarded event occurs, the circuit outputs 0. Hence an output of 1 implies that Plonky3's verifier accepts, and an honest proof is rejected only if a guarded event occurs, with probability at most $\varepsilon_{\text{guard}} \approx n \cdot 2^{-127}$ for n guarded coordinates.

Soundness of the proof system. We use Plonky3's own security assessment of each configuration as the input to our classical accounting. The proximity terms are in the Johnson list-decoding regime (Table 5). The quantum conversion in §7.1 additionally assumes round-by-round soundness for the sumcheck-based layers. At 2^{18} rows the binding term is the WHIR opening at 104.3 bits (110 bits per term, rate 1/32, folding factor 4, proof of work up to 32 bits). The other terms are constraint batching 109.0, zerocheck 114.5 and its sumcheck 113.5, bit ring switch 114.0, column batching 115.2, and commitment and transcript collisions 128 (Blake3). We write $\varepsilon_{\text{STARK}}(t)$ for the composed error against an adversary making t hash queries. Plonky3's report states it as bits of security, 104.3 at 2^{18} rows, meaning that $\varepsilon_{\text{STARK}}(t) \le t \cdot 2^{-104.3}$ for a classical adversary; §7.1 gives the quantum bound.

Authenticity of the garbling. Our scheme is privacy-free garbling [FNO15] with free-XOR [KS08] and one ciphertext per non-free gate [ZRE15], with H = Blake3 and a fresh secret Δ per garbling. Free-XOR is secure when H is circular correlation robust [CKKZ12]; we assume this of Blake3, as is standard for hash-based garbling. In the BitVM setting the garbled circuit is published at setup and the proof is chosen afterwards, so authenticity must hold for inputs chosen adaptively, after the garbled circuit is seen [BHR12a]. Let ε_{GC} be the advantage of an evaluator that sees the garbled circuit, then chooses an input x, receives its labels, and outputs the label of the value $\neg C(x)$ on the output wire. We assume that our Blake3-based free-XOR garbling has adaptive authenticity in this sense, with Blake3 modelled as a random oracle; Bellare, Hoang and Rogaway give constructions proven adaptively secure [BHR12a].
The following circuit-level statement is conditional on the Blake3 circular-correlation-robustness and adaptive-authenticity assumptions stated above. It does not cover cut-and-choose, input soldering or Bitcoin transaction enforcement.

Theorem 1. *Let C be a correctly garbled verifier of a fixed configuration, and x the encoding of a proof π for a statement.*

(i) Soundness. *If the statement is false, an evaluator holding the labels of x obtains the output label of value 0, except with probability at most $\varepsilon_{\text{STARK}}(t)$, where t bounds the hash queries of the party that chose π.*

(ii) Authenticity. *If π is honestly generated, an evaluator holding the labels of x obtains the output label of value 0 with probability at most $\varepsilon_{\text{guard}} + \varepsilon_{\text{GC}}$.*

Proof sketch. (i) If $C(x) = 1$, Plonky3's verifier accepts π by the previous paragraphs, which for a false statement happens with probability at most $\varepsilon_{\text{STARK}}(t)$. Otherwise $C(x) = 0$, and correctness of the garbling yields the label of value 0. (ii) For an honest proof $C(x) = 1$ unless a guarded

event occurs, and the label of value 0 is then the label of $\neg C(x)$, which the evaluator obtains with probability at most $\varepsilon_{\mathrm{GC}}$. □

The theorem applies only to the relation represented by C. In our experiments that relation is Keccak-f trace validity without external public values; it does not prove authorization of a bridge withdrawal.

Section 5 uses (i) to protect challengers against a false claim and (ii) to protect an honest operator.

7.1 Post-quantum analysis

The verifier construction avoids number-theoretic assumptions: the proof system, commitments, transcript and garbling use Blake3. This statement does not cover Bitcoin transaction authorization or every proposed input-publication mechanism. The estimates below are conditional on the stated random-oracle, round-by-round-soundness and adaptive-authenticity assumptions; they are not an end-to-end post-quantum security proof. Pairing-based verifiers (Groth16, Pari and their designated-verifier variants), the witness-encryption schemes built on them (§2) and Bitcoin's own Schnorr signatures fall to Shor's algorithm [Sho94]. Our verifier does not. Avoiding discrete-logarithm and pairing assumptions is necessary for quantum resistance here, but a hash-based verifier is not sufficient by itself: a quantum adversary still speeds up generic preimage and collision search [Gro96, BHT98]. Table 6 gives component-level estimates.

Fiat–Shamir against quantum adversaries. In the quantum random-oracle model, Chiesa, Manohar and Spooner [CMS19] show that applying Fiat–Shamir with a hash to a public-coin interactive oracle proof with round-by-round soundness error $\varepsilon_{\mathrm{rbr}}$ bounds the success of a quantum adversary that makes t hash queries by $O(t^2\varepsilon_{\mathrm{rbr}} + t^3/2^{\lambda})$, where $\lambda = 256$ is the digest length. A classical adversary is bounded by $O(t\,\varepsilon_{\mathrm{rbr}})$.

WHIR has round-by-round soundness [ACFY25, §5]. For the sumcheck-based layers (zerocheck, column batching, ring switch) we assume it, as is standard for sumcheck, with each of Plonky3's terms as the round error. A round with error 2^{-a} therefore offers about $a/2$ bits against a quantum adversary, and Grover search [Gro96] attains this generically when the bad challenges form an unstructured set. Proof-of-work grinding is generic search too: b bits of grinding cost 2^b hashes classically and about $2^{b/2}$ quantumly.

The proof system at our parameters. At 2^{18} rows each WHIR term targets 110 bits, of which grinding supplies 26 to 32 bits (per round: 33 queries with 30 and 28 bits of grinding, 20 with 32 and 30, 15 with 29 and 32, and 12 terminal queries with 27). Both parts halve under quantum search, so each WHIR term offers about 55 bits, and the composed 104.3 bits become about 52. The algebraic terms (zerocheck 114.5, its sumcheck 113.5, ring switch 114.0, column batching 115.2, constraint batching 109.0) are Schwartz–Zippel errors over the 2^{128}-element field and offer 54 to 58 bits. They are capped by the field: no number of queries raises them. Collisions in the 256-bit Blake3 digests cost about 2^{85} quantum queries [BHT98], and preimages 2^{128}.

The ceiling of the current configuration space. Raising Plonky3's per-term target does not raise the quantum level. We searched the target that the proven-list-decoding profile accepts at 2^{18} rows. Up to 134 bits per term it keeps the queries at $33 + 20 + 15 + 12$ and raises the grinding to 51–56 bits, its ceiling; at 200 bits it refuses, needing 103 bits of grinding. Plonky3's security report fully assesses targets up to 117 bits per term, where the composed soundness is 108.6 bits, bound by constraint batching at 109.0, a field-size term no WHIR parameter moves. Grinding and

the field-bound terms both halve under quantum search, so no configuration of the current proof system exceeds about 54 quantum bits. Reaching a quantum target is a change to the proof system, not to its parameters.

The garbling. Authenticity rests on the secrecy of Δ, and Δ can be tested against a single gate. An evaluator holding the labels a, b of an AND gate with known values x, y and its ciphertext ct checks a candidate Δ' by recomputing ct from $a, a \oplus \Delta', b$ and Δ'. A wrong candidate passes with probability 2^{-128}. Grover search therefore recovers Δ with about 2^{64} quantum evaluations of Blake3, after which every label's complement is known and the output label of the other value is free. With 128-bit labels the garbling offers about 64 bits against a quantum adversary. The same holds for a hashlock on a 128-bit output label: a preimage search over 2^{128} labels costs about 2^{64}. Grover search is highly sequential, and the adversary has only the time between setup and the end of the dispute window, but we do not rely on this.

The surrounding protocol. The protocol of §5 publishes input bits under one-time keys. Lamport, Antichain Winternitz and translated Winternitz keys rest on `HASH160` preimages. The familiar 2^{64} quantum cost for our 16-byte preimages is a single-target estimate. One Lamport key set exposes $T = 1{,}041{,}024 \approx 2^{20}$ alternative preimages, so generic multi-target search costs about $2^{128}/T \approx 2^{108}$ classically and $\sqrt{2^{128}/T} \approx 2^{54}$ quantumly. Seven independent sets lower these figures to about $2^{105.2}$ and $2^{52.6}$. Hash-chain alternatives have the same multi-target issue; Table 7 gives the widths that lift the selection itself above this bound. Even 20-byte preimages under `HASH160` are capped near 70 quantum bits for one set, rather than 80, once the target count is included. Translated Winternitz keys are further bounded by the 128-bit key chain of the translation table. Because each step of that chain is salted with the instance, the chunk and the step index, an inversion targets a single step, and its cost is 2^{128} classically and 2^{64} quantumly. The surrounding protocol is not post-quantum as instantiated. Schnorr adaptor signatures, the committee's pre-signed transactions and Taproot key-path spends fall to Shor's algorithm. Hash-based input keys avoid the adaptor-signature dependency, but their one-key-set cost assumes the unimplemented soldering in §5; the seven-key-set projection is 25–54 times the adaptor payload. Bitcoin has no opcode that natively verifies a post-quantum signature. QSB gives a non-standard hash-based authorization path under existing consensus rules [Lev26, Sta26], but whether it can replace the authorization of a pre-signed, timelocked dispute graph is outside our scope. We therefore treat graph authorization as an explicit open assumption rather than a solved component.

Parameters for a quantum target. There is no single rule that doubles every classical parameter. Fiat–Shamir and preimage-search terms need about $2\lambda_Q$ classical bits, generic collision search on an n-bit digest costs about $2^{n/3}$ quantum queries, and an information-theoretic cut-and-choose error must itself be at most $2^{-\lambda_Q}$. Seven changes follow.

- The WHIR terms need about $2\lambda_Q$ bits each. Since grinding also halves, the query count must carry most of it: from about 80 information-theoretic bits per term today to about 168 at $\lambda_Q = 100$, about 2.1× the queries. Plonky3's profile raises grinding instead, so this needs a query-heavy profile.
- The algebraic terms need challenges from a field of about $2^{2\lambda_Q+14}$ elements, which means one more tower level, $\mathbb{F}_{2^{256}}$, at $3^8 = 6{,}561$ AND gates per multiplication, three times today's. Repeating each algebraic check with independent challenges is an alternative at a similar cost.

component	attack	classical	quantum estimate	to reach λ_Q quantum bits
WHIR opening	Fiat–Shamir (FS) search, grinding	104.3 (max. 111.3)	≈ 52	$\approx 2\lambda_Q$ per term: $\approx 2.1\times$ queries at $\lambda_Q = 100$
zerocheck, sumchecks, ring switch	FS search	109–115	≈ 54–58	a field of $\gtrsim 2^{2\lambda_Q+14}$ elements: $\mathbb{F}_{2^{256}}$
Merkle and transcript hash	collision, preimage	128	≈ 85	$\gtrsim 3\lambda_Q$-bit digest and wider internal state; unmeasured
garbling (Δ, output hashlock)	Grover on 128-bit labels	128	≈ 64	$2\lambda_Q$-bit labels: 256 bits
hash-based input keys	multi-target preimage, $T \approx 2^{20}$ ($7T$ for seven sets)	≈ 108 (105.2)	≈ 54 (52.6)	at least 220 (223) bits of preimage entropy under a 256-bit hash
cut-and-choose $(181, 7)$	bad kept set	40	40	increase (N, M) for the target error
adaptor keys, pre-signed graph	Shor	–	broken	post-quantum transaction authorization

Table 6: The measured proof system has a bottleneck near 52 quantum bits under the stated assumptions; this is not the composed protocol level. The reported cut-and-choose parameters provide about 40 statistical bits, and the transaction authorization is broken by Shor. Reaching 100 quantum bits also requires more queries, a 256-bit field, 256-bit labels, a hash with at least a 300-bit digest and sufficiently wide internal state, a 256-bit input hash with 28-byte preimages, stronger cut-and-choose and post-quantum transaction authorization.

- Labels of $2\lambda_Q$ bits: 256-bit labels double the ciphertext size to 32 bytes per non-free gate. The gate count is unchanged, because labels feed the garbling hash, not the circuit.
- Merkle commitments and the transcript need a hash that withstands 2^{λ_Q} quantum collision search. Generic collision search therefore calls for at least a $3\lambda_Q$-bit digest and a sufficiently wide internal state. Blake3-256 caps this component near 85 quantum bits; requesting a longer XOF output does not widen its 256-bit chaining value. A replacement hash circuit and its cost remain to be designed and measured.
- For T possible input-authentication targets, 100 quantum bits require at least $200 + \lceil \log_2 T \rceil$ bits of preimage entropy under a hash wider than 160 bits: about 220 bits for one key set and 223 for seven. Byte-aligned 28-byte preimages under `SHA256` meet this generic target, but that construction and its larger witness cost remain to be designed and measured.
- Cut-and-choose needs parameters satisfying $1/\binom{N}{M} \leq 2^{-\lambda_Q}$, together with an ungrindable uniform selection of the opened instances. The setup time and kept-instance storage must be recomputed for those parameters.
- The pre-signed graph needs post-quantum transaction authorization. Bitcoin currently has no general post-quantum signature opcode. QSB gives a non-standard hash-based spend under existing consensus rules, but adapting it to the committee-authorized, timelocked graph is unstudied here.

Applying only the first three factors to the measured 2^{18} verifier profile gives a partial lower-bound estimate, not a 100-bit configuration measurement. Merkle paths and leaf hashes scale with the

query count (2.1×), and leaves also with the element size (2×), to about 45M gates. The transcripts absorb twice the bytes, about 24M. Multiplications triple, and those that depend on the queries also scale with them, to about 213M. The result is about 280M non-free gates, 3× today's, and about 9 GB garbled at 32 bytes per gate. It leaves Blake3-256 and its 85-bit quantum collision ceiling unchanged and excludes the replacement hash circuit, stronger cut-and-choose, input authentication and graph authorization. It is therefore not an estimate of a 100-bit end-to-end protocol.

8 Discussion and conclusion

We have shown that the verification path exercised by one no-public-input Keccak-f configuration of Plonky3's Boolean-WHIR pipeline fits garbling well. At 2^{18} rows it has 94.1M non-free gates and a 1.5 GB garbled artifact, about 30× fewer non-free gates than the reported Boolean-garbled Groth16 verifier, and the input shrinks further when the statement is itself a recursion proof: the verifier of the last of three narrow-recursion proofs of Ziren's binary stage takes 564,552 input bits and 218.0M non-free gates, measured on real proofs but resting on unreviewed changes to Plonky3 and Ziren, with a run that binds neither the zkVM program nor its outputs. This is a verifier-engineering result, not an end-to-end Bitcoin protocol, and the rest of this section says what stands between the two.

Input authentication dominates the measured components. The largest measured on-chain component is revealing and authenticating the 1,041,024 circuit-input bits in a dispute, not committing or transporting them. The raw bits fit a 33-kvB transport fixture, whereas authenticating them for one key set costs 2.22–17.27 MvB (Table 3); strict Core 31.1 accepted the four signed funding–reveal slices, but only in confirmed stages, none is an atomic unconfirmed graph, and only the Lamport fixture also tests a join. The hash-based rows assume the unimplemented soldering or multiply by M, and the independent seven-set projection is 55–121 MvB before the transaction graph. WHIR tuning removes only a fifth of the input (Table 10); equal-area narrow-statement models remove 37–58% but need a new AIR; batching amortizes the input per permutation by 3.76× without reducing the absolute input; and narrow recursion reaches 0.54× (Table 11) at the price of a recursion prover and of proof-system changes still to be reviewed. The approximately 52-bit quantum figure applies to the measured proof-system parameters under the stated assumptions, cut-and-choose is configured at 40 statistical bits, and the current transaction authorization is not post-quantum. Before deployment, the construction needs public-input support, an implemented and measured multi-copy input layer, a serialized and policy-tested Assert graph, post-quantum graph authorization, and a measured parameter set with an enlarged field and labels, a wider-state hash and stronger cut-and-choose.

Open input-layer directions. TinyLabels compresses honest selected-label delivery to $|x| + \mathrm{poly}(\lambda)$ online bits under Ring-LWE in the random-oracle model [DLL24]; its reported batch of 699,050 128-bit labels with an approximately 56-KB selector key would cover the present input in two batches, roughly 0.24 MB before Bitcoin encoding. The selector key is not publicly verifiable, however, and the malicious-2PC wrapper turns malformed delivery into an abort rather than public evidence, so a withheld or malicious key leaves the evaluator with garbage labels and no reject label that opens Disprove; a publicly verifiable or penalizable Batch-Select, consistent across the M kept garblings, is open. Segmented verification would instead commit once to a fixed serialization of the proof, locate a faulty verifier segment by bisection and authenticate only the challenged chunk: for N = 1,041,024 bits in chunks of at most B bits under a 256-bit Merkle tree, the data-only

challenged input is $B + 256\lceil \log_2 \lceil N/B \rceil \rceil + C_{\text{state}}$, where C_{state} binds the transcript, accumulator and boundary state; a 6,656-bit first-round query chunk gives 157 chunks, an eight-digest path and 8,704 bits before state. Query segmentation alone is insufficient, since leaf rows and paths are 510,464 of the input bits and the remaining 530,560 are global, so the opened rows, sumchecks, ring switch and transcript must be segmented and their boundary states bound, with a fault-location and withholding-penalty protocol; none is implemented, and the worst-case disclosure remains the full N bits.

Reducing the circuit. Our sizes are for the 1,625-column Keccak AIR; Plonky3's binary SHA-256 and Blake3 AIRs read no next-row columns, which our verifier assumes, and supporting them changes the opening layout. The column-dependent terms, about a third of the circuit, scale with the AIR's width. The ring switch closing, another third, is the first target, and an $\mathbb{F}_{2^{128}}$ multiplier below Karatsuba's 3^7 the second: bitvm-gc's $\mathbb{F}_{2^{233}}$ multiplier, evaluating, multiplying and interpolating over $\mathbb{F}_{2^9}$, costs 2,097 AND gates for a field almost twice as wide, at the price of 1.5M XORs per product. Per-tree Merkle cap heights and a narrower AIR reduce the hashing and column terms; the recursion of Table 11 does both, with a preprocessed tree read deep below its root and tables whose opened values are linear forms of their columns. At the other end of the graph, a Core-tested false-label-derived Taproot pointlock reduces the 16-byte-label Disprove spend from 97 to 83 vB when both outputs retain the same timeout sibling, a 14-vB micro-optimization that adds the discrete-logarithm assumption, does not reveal the false label and cannot by itself constrain the payout outputs.

References

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: Reed–solomon proximity testing with fewer queries. In *Advances in Cryptology – CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 380–413. Springer, 2024. Full version: Cryptology ePrint Archive, Paper 2024/390.

[ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon proximity testing with super-fast verification. In *Advances in Cryptology – EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 214–243. Springer, 2025. Cryptology ePrint Archive, Paper 2024/1586.

[ACG+26] Michel Abdalla, Brent Carmer, Muhammed El Gebali, Handan Kilinc-Alper, Mikhail Komarov, Yaroslav Rebenko, Lev Soukhanov, Erkan Tairi, Elena Tatuzova, and Patrick Towa. Bitcoin PIPEs v2: Covenants and ZKPs on Bitcoin via witness encryption. Cryptology ePrint Archive, Paper 2026/186, 2026.

[BHR12a] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Adaptively secure garbling with applications to one-time programs and secure outsourcing. In *Advances in Cryptology – ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 134–153. Springer, 2012.

[BHR12b] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, pages 784–796. ACM, 2012.

[BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN 1998: Theoretical Informatics*, volume 1380 of *LNCS*, pages 163–169. Springer, 1998.

[BIJ+20] James Bartusek, Yuval Ishai, Aayush Jain, Fermi Ma, Amit Sahai, and Mark Zhandry. Affine determinant programs: A framework for obfuscation and witness encryption. In *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, volume 151 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 82:1–82:39. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020.

[Bit25] BitVM. Garbled SNARK verifier. Source code, commit `5a2cd4dc6cb19e37adb1b3ab94414e01d1e8b338`, 2025.

[Bit26] Bitcoin Core developers. Bitcoin Core 31.1 transaction relay policy constants. Source code, release tag `v31.1`, 2026. Defines `MAX_STANDARD_TX_WEIGHT` and `DEFAULT_CLUSTER_SIZE_LIMIT_KVB`.

[Can89] David G. Cantor. On arithmetical algorithms over finite fields. *Journal of Combinatorial Theory, Series A*, 50(2):285–300, 1989.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Advances in Cryptology – EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 499–530. Springer, 2023. Full version: Cryptology ePrint Archive, Paper 2022/1355.

[Che25] Weikeng Chen. SoK: BitVM with succinct on-chain cost. Cryptology ePrint Archive, Paper 2025/1253, 2025.

[CKKZ12] Seung Geol Choi, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. On the security of the "free-XOR" technique. In *Theory of Cryptography Conference (TCC 2012)*, volume 7194 of *LNCS*, pages 39–53. Springer, 2012.

[CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. Succinct arguments in the quantum random oracle model. In *Theory of Cryptography Conference (TCC 2019), Part II*, volume 11892 of *LNCS*, pages 1–29. Springer, 2019. Cryptology ePrint Archive, Paper 2019/834.

[DLL24] Marian Dietz, Hanjun Li, and Huijia Lin. TinyLabels: How to compress garbled circuit input labels, efficiently. Cryptology ePrint Archive, Paper 2024/2048, 2024.

[DMS24] Michel Dellepere, Pratyush Mishra, and Alireza Shirzad. Garuda and pari: Faster and smaller SNARKs via equifficient polynomial commitments. Cryptology ePrint Archive, Paper 2024/1245, 2024.

[DP25] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. In *Advances in Cryptology – EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 93–122. Springer, 2025. Full version: Cryptology ePrint Archive, Paper 2023/1784.

[DP26] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. In *Advances in Cryptology – EUROCRYPT 2026, Part VII*, volume 16547 of *LNCS*, pages 3–32. Springer, 2026. Full version: Cryptology ePrint Archive, Paper 2024/504.

[DYH+26] Stephen Duan, Jingping Yin, Blake Hou, Vanhger Pham, Ethan Zhu, and Ming Guo. Ziren: Succinct arguments for MIPS32 execution. Cryptology ePrint Archive, Paper 2026/2330, 2026.

[Eag25] Liam Eagen. Glock: Garbled locks for Bitcoin. Cryptology ePrint Archive, Paper 2025/1485, 2025.

[EL26] Liam Eagen and Ying Tong Lai. Argo MAC: Garbling with elliptic curve MACs. Cryptology ePrint Archive, Paper 2026/049, 2026.

[FBF+26] Ariel Futoransky, Fadi Barbàra, Ramses Fernandez, Gabriel Larotonda, and Sergio Demian Lerner. TRAPGC-DV: Trapdoor garbled circuit designated verification of Groth16 on Bitcoin. Cryptology ePrint Archive, Paper 2026/1430, 2026.

[FDZ25] Alva Fu, Stephen Duan, and Ethan Zhu. Instantiating BitVM3 from label forward propagation. `https://www.goat.network/bitvm3-label-forward-propagation`, 2025.

[FLB25] Ariel Futoransky, Gabriel Larotonda, and Fadi Barbara. A note on the security of the BitVM3 garbling scheme. Cryptology ePrint Archive, Paper 2025/1291, 2025.

[FNO15] Tore Kasper Frederiksen, Jesper Buus Nielsen, and Claudio Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 191–219. Springer, 2015.

[GGSW13] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In *45th ACM STOC*, pages 467–476. ACM, 2013.

[GKPW24] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In *Advances in Cryptology – CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 352–386. Springer, 2024.

[GKS+26] Sanjam Garg, Dimitris Kolonelos, Mikhail Sergeevitch, Srivatsan Sridhar, and David Tse. BABE: Verifying proofs on Bitcoin made 1000x cheaper. Cryptology ePrint Archive, Paper 2026/065, 2026.

[GOA26a] GOAT Network. bitcoin-stark-verifier: a STARK verifier in Bitcoin Script without OP_CAT. GOATNetwork/bitcoin-stark-verifier, crates `poseidon2` and `whir`, commit `d45e453`, 2026.

[GOA26b] GOAT Network. bitvm-gc: garbled SNARK verifier and BABE for BitVM. Source code, commit `10083d4a63eb08a344dc40ee84bc11394b218a89`, 2026.

[Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219. ACM, 1996.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, 2016.

[KFLT26] Nakul Khambhati, Aaron Feickert, Christian Lewe, and Mukesh Tiwari. Garbling Groth16 with native group operations. Cryptology ePrint Archive, Paper 2026/2100, 2026.

[KLN25] Victor I. Kolobov, Avihu M. Levy, and Moni Naor. ColliderVM: Stateful computation on Bitcoin without fraud proofs. Cryptology ePrint Archive, Paper 2025/591, 2025.

[KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In *Automata, Languages and Programming (ICALP 2008), Part II*, volume 5126 of *LNCS*, pages 486–498. Springer, 2008.

[KTA+26] Nakul Khambhati, Mukesh Tiwari, Azz, Sapin Bajracharya, Manish Bista, Liam Eagen, Christian Lewe, and Aaron Feickert. Mosaic: Practical malicious security for garbled circuits on Bitcoin. Cryptology ePrint Archive, Paper 2026/812, 2026.

[LAA+25] Robin Linus, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, Andrea Pelosi, Orfeas Stefanos Thyfronitis Litos, Christos Stefo, David Tse, and Alexei Zamyatin. Bridging Bitcoin to second layers via BitVM2. Cryptology ePrint Archive, Paper 2025/1158, 2025.

[LAM+24] Sergio Demian Lerner, Ramon Amela, Shreemoy Mishra, Martin Jonas, and Javier Álvarez Cid-Fuentes. BitVMX: A CPU for universal computation on Bitcoin. arXiv preprint arXiv:2405.06842, 2024.

[LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to Reed–Solomon erasure codes. In *IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 316–325. IEEE, 2014.

[Lev26] Avihu Mordechai Levy. Quantum-safe Bitcoin transactions without softforks. Technical report, `https://github.com/avihu28/Quantum-Safe-Bitcoin-Transactions/blob/main/paper/QSB.pdf`, 2026. Source code: `https://github.com/avihu28/Quantum-Safe-Bitcoin-Transactions`.

[LFKN92] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992.

[Lin25] Robin Linus. BitVM3: Efficient computation on Bitcoin. `https://bitvm.org/bitvm3-rsa.pdf`, 2025.

[LJF25] Sergio Demian Lerner, Martin Jonas, and Ariel Futoransky. ESSPI: ECDSA/Schnorr signed program input for BitVMX. arXiv preprint arXiv:2503.02772, 2025.

[Nat15] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS PUB 202, NIST, 2015.

[OANWO20] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. `https://github.com/BLAKE3-team/BLAKE3-specs`, 2020.

[Pha26] Vanhger Pham. Deferred binding: Extending BABE for dynamic public inputs in GOAT BitVM3. GOAT Research, HackMD, 2026. Accessed 5 November 2026. Implementation: GOATNetwork/bitvm2-gc, branch `feat/goat-bitvm3`, commit `c23130a`.

[Plo26] Plonky3 contributors. Plonky3: binary tower fields, WHIR and multi-STARK. Source code, commit `ca4da2daa2e0dfdd31419af5c870813083398955`, 2026.

[Rub25] Jeremy Rubin. Delbrag. Technical report, first written November 2024, 2025.

[Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.

[SRG+26] Lev Soukhanov, Yaroslav Rebenko, Muhammad El Gebali, Mikhail Komarov, Handan Kilinc-Alper, Michel Abdalla, and Patrick Towa. Implementable witness encryption from arithmetic affine determinant programs. Cryptology ePrint Archive, Paper 2026/175, 2026.

[SST+26] Mikhail Sergeevitch, Konrad Staniec, David Tse, Nikhil Vanjani, and Robin Linus Woll. Non-interactive translation of Winternitz signatures to Lamport signatures via secret sharing. Cryptology ePrint Archive, Paper 2026/1684, 2026.

[Sta26] StarkWare. The first quantum-safe Bitcoin transaction has been mined. `https://starkware.co/blog/the-first-quantum-safe-bitcoin-transaction-has-been-mined/`, August 2026. Mainnet transaction: `https://mempool.space/tx/305a24ffea912b9cf428f29ebf952321c96dab5bab284fc0d0801562f5abab07`.

[TF26a] Mukesh Tiwari and Aaron Feickert. Antichain Winternitz: Guaranteed garbled-circuit label revelation on Bitcoin with permissionless recovery. Cryptology ePrint Archive, Paper 2026/1568, 2026.

[TF26b] Mukesh Tiwari and Aaron Feickert. The post-quantum cost of garbled-circuit Bitcoin bridges. Cryptology ePrint Archive, Paper 2026/2383, 2026.

[WAA+26] Robin Linus Woll, Ioannis Alexopoulos, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and David Tse. BitVM3: Efficient Bitcoin bridges via garbled circuits. Cryptology ePrint Archive, Paper 2026/933, 2026.

[Wie88] Doug Wiedemann. An iterated quadratic extension of GF(2). *The Fibonacci Quarterly*, 26(4):290–295, 1988.

[WNT20] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP 342: Validation of Taproot scripts. Bitcoin Improvement Proposal, 2020. Defines Tapscript signature semantics and resource limits.

[ZKM26] ZKM. Ziren: a MIPS zero-knowledge virtual machine, branch `feat/binary-whir-blake3`. Source code, commit `80af2779c17d4aa40ba2cafdea21734812892b4e`, 2026.

[ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. In *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250. Springer, 2015.

A Bitcoin integration details

This appendix holds the material that §5 summarizes: the hash-based label selection, its post-quantum instantiation, the four input-publication schemes as fixtures, and the executed dispute components.

A.1 Hash-based label selection

Figure 5 is the hash-based counterpart of Figure 3: the signature completion is replaced by a pair of hash-chain openings, the evaluator contributes nothing at setup, and the transaction binding must be added as a separate signature.

A.2 A post-quantum instantiation

A post-quantum instantiation. Figure 5 becomes a post-quantum replacement for the adaptor under three changes, none of which is available on Bitcoin today. First, the forward hashing moves out of Script into a native batch verifier: one leaf per transaction carries the batch root ρ, the first digit index and the count, and an opcode re-derives every endpoint from the records on the stack. Second, the transaction-binding signature becomes hash-based; the draft P2MR output type and its 0xC2 leaf version[1] would provide a 548-byte stateful hash-based signature per transaction and 6,000-byte stack items. Third, the node width n must match the quantum target: forging a different digit from a published opening is a second-preimage search over the roughly 2^{22} chain nodes that the whole input makes public, so $n = 16$ gives about 53 quantum bits, $n = 24$ about 85 and $n = 32$ about 117.

We serialized this construction exactly with rust-bitcoin for $n = 16$ and $n = 24$ at $k = 4$, including the P2MR outputs, 0xC2 leaves, placeholder signatures, depth-one control blocks and canonical multi-record stack items, and tested the pair code and the batch parser against tampering, reordering, cross-batch splicing and padding; nothing in it is executable by Bitcoin Core. Table 7 gives the results together with a projection whose model reproduces both serialized fixtures within 0.03%. The operative row is $n = 32$, $k = 8$: full-width nodes, about 117 quantum bits for the selection, and the same 2.13 MvB as the Schnorr adaptor, at the price of 255 hash steps per byte inside the opcode, about 3.3×10^7 compressions for the input. Its setup is two seeds per digit rather than the adaptor's 2^k secrets, so widening k does not grow the choice table; only the translation table grows, to $2^k \cdot k \cdot 16$ bytes per digit, 4.3 GB per instance at $k = 8$. The delivery and soldering of that table across the M kept garblings is the same open item as for every other row.

A.3 The four input schemes

The four one-time schemes of Table 3 trade on-chain size against assumptions.

- *Lamport.* Each bit has two hashed 16-byte preimages, and the revealed preimage is the one-time-signature witness. The leaf script per bit,

  `OP_HASH160 OP_DUP` h_0 `OP_EQUAL OP_SWAP` h_1 `OP_EQUAL OP_BOOLOR OP_VERIFY`

  is 49 bytes, and the witness 17 bytes. We executed it with 998 bits per input, and it rejected wrong preimages. The transaction-bound fixture prefixes each leaf with a 32-byte authorization key and `OP_CHECKSIGVERIFY`, and supplies a 64-byte `SIGHASH_DEFAULT` signature. Its

[1] BIP360, `https://github.com/bitcoin/bips/blob/master/bip-0360.mediawiki`, and the SHRINCS proposal for a stateful hash-based signature in that leaf; both are drafts and may change.

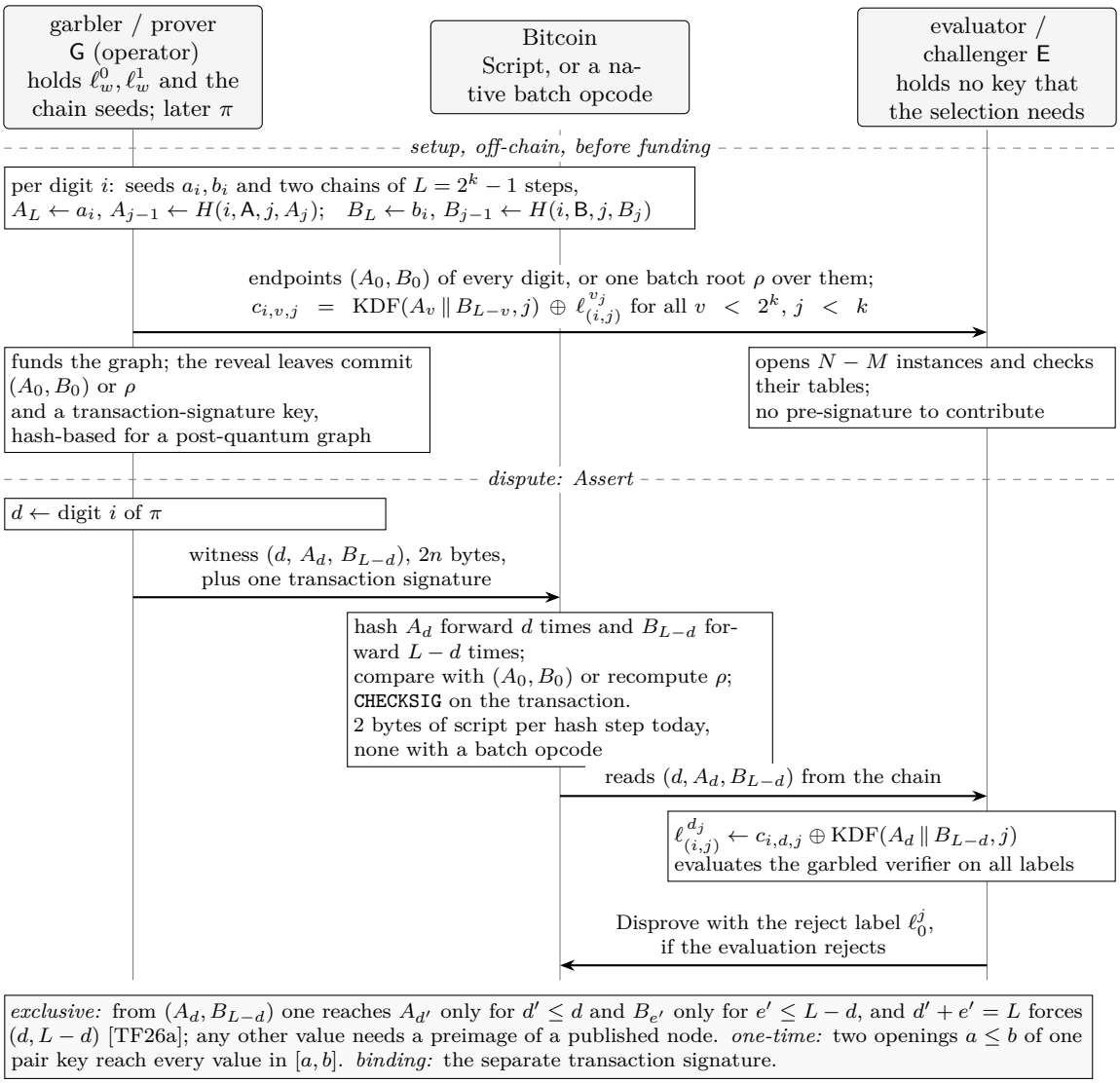


Figure 5: Hash-based label selection for one k-bit digit position i with n-byte chain nodes. The selection material is two nodes instead of one signature, the evaluator contributes nothing at setup, and the transaction binding that the adaptor gets from the same signature must be added separately. Verification is a hash ladder, in Script today or in a batch opcode under the soft fork of Table 7.

selection	consensus	n	k	vB/bit	component	selection security, quantum
Schnorr adaptor (Fig. 3)	current	–	8	2.14	2.23 MvB	broken (Shor)
WOTS translation [SST+26] in Script	current	20	3	7.59	7.90 MvB	≈ 64 bits (κ chain; nodes ≈ 70)
Antichain $(2, 16)$ in Script (Fig. 5)	current	16	4	11.41	11.88 MvB	≈ 53 bits
batched selector, one record per item	P2MR, 0xC2, batch opcode	16	4	2.05	2.13 MvB	≈ 53 bits
batched selector, 11-record items	same	16	4	2.04	2.12 MvB	≈ 53 bits
batched selector, 5-record items	same	24	4	3.05	3.17 MvB	≈ 85 bits
batched selector, projected	same	32	8	≈ 2.05	≈ 2.13 MvB	≈ 117 bits

Table 7: Hash-based label selection against the adaptor baseline for the 1,041,024-bit input, funding plus reveals. The first three rows are the Core-tested fixtures of Table 3. The batched-selector rows are exact rust-bitcoin serializations of a construction that requires the draft P2MR output, a hash-based transaction signature and a hypothetical batch verification opcode; the last row is a model projection. The security column is the multi-target second-preimage cost of the chain nodes alone, except for WOTS translation, whose 128-bit key chain gives the lower bound; the composed protocol remains bound by the proof system (§7.1).

initial stack has 999 items and the measured transient peak is exactly the 1,000-item Tapscript limit [WNT20]. Its serialized size is the Lamport row of Table 3. The fixture uses a one-leaf tree with a 33-byte control block; timeout siblings and their longer control paths belong to the missing complete graph.

- *Antichain Winternitz.* Plain Winternitz chains are unsafe here: from a published chain value the evaluator hashes forward to the labels of other digit values [TF26a]. Antichain Winternitz closes this and costs a cited marginal 11.31 vB per bit at parameters $(2, 16)$, counting the hash-chain opening witness and authentication script but cancelling fixed transaction overhead. Our transaction-bound fixture pins the upstream implementation, prepends a 64-byte Schnorr authorization, and includes two pay-to-Taproot (P2TR) outputs, input shells, leaf scripts and depth-zero controls. With at most 332 four-bit digits per input, its size is the Antichain row of Table 3, an all-nonzero worst case that zero-valued ScriptNum coordinates only reduce. The source reports 0.256 kB of off-chain table per bit. Its larger parameters save more on-chain but need up to 43.8 kB per bit off-chain [TF26a], 46 GB for our input.

- *Winternitz with a translation table.* The translation gadget of [SST+26] also closes the gap of plain Winternitz: each label is a Shamir secret whose threshold is set by the chunk's checksum and the bit value, so a signed chunk opens exactly the labels of its bits. In the source, one such Winternitz signature over 508 bits, its Assert, is 3,748 vB, or 7.38 vB per bit. Our transaction-bound fixture uses 3-bit digits, eight per chunk, with the checksum split over two chains, and the authorization, outputs and controls of the Antichain fixture. With at most 49 chunks per input, its size is the WOTS row of Table 3, again an all-nonzero worst case. The price is the off-chain table, 1,830 bytes per bit or 1.905 GB per instance. The challenger must decode digits as consensus does and, at setup, reject keys in which one chain end lies within seven hashes of another; neither check costs on-chain space.

- *Schnorr adaptor signatures* over 8-bit digits, as in BitVM3 and Mosaic. BitVM3 reports 9,707 weight units (WU) for a 1,024-bit Assert transaction with one input and two P2TR outputs [WAA+26], or 2.37 vB per bit at that size. Its printed witness formula instead sums to 9,451 WU once the SegWit marker and flag are included, while the merged 64-byte-`SIGHASH_DEFAULT` implementation implies 9,323 WU for that shell. Our executable fixture reproduces 9,323 WU, uses the BIP342 opcode index rather than a byte offset for every code separator, and reaches the exact 998-signature stack capacity. Core 31.1 validates all completed signatures. Its size for the 130,128 eight-bit digits is the adaptor row of Table 3. Mosaic's source construction solders all kept instances to one set of signatures by polynomial label correlation, with an on-chain footprint independent of the number of instances [KTA+26]. This repository tests only the one-key-set transaction fixture; it does not implement or test Mosaic's polynomial correlation or setup verification. The signatures rest on discrete logarithms, and an evaluator that holds the signing key can spend without completing the adaptor [TF26a].

Packing more bits into each adaptor completion reduces the serialized transaction shape. Our Core-tested 16-bit fixture uses 2,960 vB of funding and 1,111,128 vB across 12 reveals, or 1,114,088 vB in 13 transactions. Exact serializations of 10-, 11- and 12-bit digits, still with placeholder signatures and not replayed in Core, give 1,782,498, 1,620,499 and 1,485,410 vB including funding. The price is off-chain: a table of every digit's 2^b choices at 65 bytes each is 2.165 GB per key set at 8 bits, 6.929, 12.598 and 23.097 GB at 10, 11 and 12 bits, and 277.162 GB at 16. This is an encoding sensitivity result, not yet a binary-label authentication mechanism: the complete 2^{16} choice table and exclusive digit-to-binary-label delivery remain unimplemented. We therefore retain the 8-bit construction as the authenticated baseline in Table 3.

A.4 Executed components

Executed components. We ran the dispute components on a small circuit, with its on-chain steps executed as Bitcoin Script by `bitcoin-scriptexec`. The statement is knowledge of a 64-byte x with Blake3(x) = h: one Blake3 compression, 512 input bits and 10,536 non-free gates. The operator garbles it at setup, before choosing x, and publishes Lamport hashes, translation ciphertexts, the labels of the two constant wires and SHA256 of the reject label. Its Assert publication runs through a 25 KB Lamport script under the 1,000-item stack limit. The challenger derives the input labels from the published preimages and evaluates the stored garbling without Δ or any label it was not handed. For a true x it obtains the accept label, which the Disprove hashlock script rejects; for x with one bit changed it obtains the reject label, which opens it. Assert rejects a preimage outside the committed keys. We then ran the same dispute on the Keccak-f STARK verifier at 2^5 rows. Garbled with every input at 0, it reads 586,240 input bits, which the operator publishes through 588 Assert scripts of at most 998 bits each, 28.7 MB of script in all, each executed under the stack limit. The challenger's evaluation of the stored garbling from the published labels gives the accept label on the real proof and the reject label, which opens Disprove, on the proof with one opened value changed. Signatures, timelocks and cut-and-choose are not part of either run. These runs validate individual input-publication, label-translation, evaluation and hashlock components for one garbling. They do not validate Figure 2, multi-instance soldering, transaction policy or ledger-level security.

phase	non-free gates	share
ring switch closing	31,633,960	33.6%
WHIR Merkle paths	14,613,722	15.5%
column combination	11,582,352	12.3%
AIR constraints (1,650, Horner under α)	10,736,110	11.4%
STARK transcript	10,531,194	11.2%
WHIR leaf hashes	3,320,880	3.5%
WHIR closing query weights	3,069,960	3.3%
WHIR query folds	2,624,400	2.8%
WHIR transcript	1,603,940	1.7%
zerocheck sumcheck	1,227,336	1.3%
ring switch statement	835,688	0.9%
other (9 phases)	2,290,006	2.4%
total	94,069,548	100%

Table 8: A third of the 2^{18}-row verifier is the ring switch closing, and the five largest phases make up 84%. Non-free gates by the verifier phase that emits them.

B Additional measurements

B.1 Artifact availability

Artifact availability. The source repository includes the locked dependencies and benchmark commands; the commands and memory caps are listed in `whir-gc/README.md`. The raw measurement logs, with a SHA-256 manifest and a map from tables and figures to logs, are archived separately and available on request. The experimental Plonky3 and Ziren changes behind Table 11 are Ziren commit `80af2779` on `feat/binary-whir-blake3` and Plonky3 commit `be1c6f92` on ProjectZKM's `zkm/binary-whir`; `patches/README.md` lists them and the commands that rebuild the tapes. `Cargo.lock` pins Plonky3 at revision `ca4da2daa2e0dfdd31419af5c870813083398955` and bitvm-gc at revision `10083d4a63eb08a344dc40ee84bc11394b218a89`. Gate, wire and input counts are deterministic; wall-clock times are single-run engineering measurements on the machines above unless a range is stated.

B.2 Gate profile, WHIR parameters and ablation

Table 8 breaks the 2^{18}-row verifier down by the phase that emits each gate, and Table 9 isolates two of our choices on the measured verifier.

WHIR alone, and its parameters. On a standalone instance, garbling only the WHIR opening of an 18-variable polynomial (rate 1/32, folding 4, $35 + 22 + 16$ queries) cost 24.3M non-free gates with single-root commitments, and 21.2M with a Merkle cap of 32 roots. That cap is Plonky3's recommended height and was the optimum in our sweep of 1 to 256 roots. Three quarters of the WHIR circuit was hashing. Across rates 1/16–1/64 and folding factors 2–5, rate 1/32 with folding 4 gave the smallest circuit among the configurations we kept; rate 1/64 saved a further 10% of gates at the price of a prover domain twice as large.

configuration	non-free gates	peak memory	time
2^5 rows, this work (streamed)	44,573,121	0.69 GiB	33 s (plan + garble)
2^5 rows, stored build	44,573,121	12.4 GiB	283 s (build only)
2^5 rows, original Blake3 adder	58,997,590 (+32%)	–	–
2^8 rows, this work	66,459,859	–	–
2^8 rows, original Blake3 adder	85,768,603 (+29%)	–	–

Table 9: The one-AND adder removes about a quarter of the verifier's gates, and streaming cuts memory 18×. Peak memory is the whole process's, including proof generation, which is negligible at 2^5 rows; the stored build's time excludes garbling.

B.3 Core replay and fee equivalents

The raw transport fixture of Table 3 serializes to 132,788 bytes, 133,193 WU and 33,299 vB, below the 400,000-WU standard-transaction limit [Bit26]; it uses synthetic outpoints and was not submitted to `testmempoolaccept`. The signed-envelope fixture commits the 130,128-byte payload in a 154-vB wallet-signed transaction and reveals it in a 32,850-vB script-path spend whose 130,917-byte tapscript, control block and transaction shells are included; strict Core 31.1 policy accepts the unconfirmed parent–child pair. The adaptor fixture serializes 133 P2TR inputs in 23 two-output transactions, the largest 399,936 WU, so its packing is specific to the measured two-output shell.

We exported all four transaction-bound fixtures to isolated Bitcoin Core 31.1 regtest nodes with non-standard transactions disabled. Core rejected each representative reveal while its funding parent was unconfirmed as `too-large-cluster`. After funding confirmation, it accepted every reveal. The adaptor fixture used a 5,841-vB funding transaction and 23 reveals totaling 2,222,213 vB, or 2,228,054 vB together, and mined in 3 blocks. WOTS translation used 38,211 vB of funding and 81 reveals totaling 7,861,144 vB, or 7,899,355 vB together, and mined in 9 blocks; a reveal with one revealed chain node altered failed its script.

Antichain used 33,836 vB of funding and 131 reveals totaling 11,848,875 vB, or 11,882,711 vB together, and mined in 12 blocks. Lamport used 45,016 vB of funding and 174 reveals totaling 17,265,635 vB, or 17,310,651 vB together, and mined in 18 blocks; 173 reveals are 99,312 vB (397,246 WU) and the tail is 84,659 vB. Core also rejected Lamport's signed 174-parent join while its reveal parents were unconfirmed. After they confirmed, it accepted and mined the 13,060-vB join, bringing that tested slice to 17,323,711 vB. This establishes staged standard-policy feasibility for these bounded signed publication slices, not peer-to-peer propagation, robust fee management, multi-leaf timeout trees or the missing challenge, Disprove, anchor and soldering graph. Adaptor, WOTS-translation and Antichain join/completion transactions remain unconstructed.

At BABE's historical scenario of 2.2 sat/vB and $95,500 per bitcoin [GKS+26], the one-set measurements have component fee equivalents of 0.049, 0.173, 0.261 and 0.380 BTC for the adaptor, WOTS translation, Antichain and Lamport, respectively; these are neither current quotes nor fees for a complete dispute. Their serialized packings use 23, 81, 131 and 174 transactions; dividing by 100 kvB instead gives 23, 79, 119 and 173 capacity equivalents, which are not transaction counts. For the adaptor fixture, the two extra inputs of the 133-input packing remove three transaction shells and lower the reveal total by 184 vB, while creating the two extra P2TR prevouts costs about 86 vB, so that packing remains smaller before counting its three fewer completion parents. For example, 1,041,024 Lamport bits require 1,044 inputs at 998 bits per input, and the simplest one-output 400-kWU packing fits at most six full inputs, so the minimal-input, maximal-chunk layout uses 174 transactions. This is not a transaction-count lower bound: repartitioning into more, smaller

component	measured	tuned
opened values (current and next row)	416,000	416,000
zerocheck and ring switch	64,256	64,256
WHIR Merkle paths	346,624	239,616
WHIR leaf rows	163,840	92,160
WHIR caps, OOD answers, sumchecks, final polynomial	50,304	33,920
total input bits	1,041,024	845,952
STIR queries per round	$33 + 20 + 15 + 12$	$16 + 12 + 9 + 8$
rate, folding, grinding budget	1/32, 4, 32	1/256, 4, 48
composed soundness (bits)	104.3	103.9

Table 10: Tuning WHIR removes a fifth of the input, and the AIR's opened values set the floor. Input bits at 2^{18} rows by proof component, for the measured configuration and the smallest configuration found in the sweep; 128 bits per field element and 256 per digest, with full Merkle paths below each cap.

inputs can fit 173 transactions, but increases the total serialized footprint. The strict Core fixture also shows that mempool policy cannot admit the 174 reveals as a single zero-confirmation graph: funding must confirm first, the independent reveals then require at least 18 blocks, and only then can the join be admitted. The tuned proof of Table 10 reduces input-proportional terms by 18.74%. A deployment must serialize and policy-test the tuned transactions and size its challenge bond to the resulting complete graph.

B.4 Input sweep

Table 10 gives the input bits of the 2^{18}-row verifier by proof component for the measured configuration and for the smallest configuration in the sweep of §6. The equal-area narrow-statement models of §6, widths 128 and 32 at heights 2^{22} and 2^{24}, reduce the input by 36.6% and 38.9% under the current WHIR profile and by 55.4% and 57.6% with the best swept configuration; the power-of-two rounding represents 126.03% of the original cell count, and the aggressive result inherits the quantum-search and prover-cost caveats of grinding. Recursion into a proof whose own WHIR opening is still some 0.37M bits at these parameters is the other route below the floor.

B.5 Narrow recursion

Table 11 lists every verifier of the narrow-recursion experiment of §6; from the third row on, each is a level-4 proof re-proved from the same level-3 tape at the Johnson rate 1/1024 and re-recorded. The tape machine can bind up to eight of a tape's first inputs as public values, enough for the binary stage's public-value digest, which commits to the zkVM program's verifying key and outputs; our runs bind none. We translate a tape gate by gate in the `binary_tape` module of `whir-gc`, and Ziren's `binary-recursion` crate records the tapes.

The steps combine recording choices with changes to the proof system. The recorder reads opened values the verifier already knows (padding and constant preprocessed columns) as constants, and reads the fixed preprocessed Merkle tree, down to 18 layers below its root, as circuit constants selected by a one-hot of the query's top index bits. The schedule folds 4, 5, 4 and 4 variables per WHIR round at 46 bits of grinding. The five protocol changes are: (i) ring-switch claims at points sharing the coordinates that one packed element absorbs send a single tensor, weighted by powers of a challenge drawn after the points, and a pair's main and preprocessed switches run as one over

verifier	input bits	non-free gates	garbled
binary stage (level 1)	18,050,304	3,371.4M	53.94 GB
narrow recursion, level 2, Ziren `9398469a`	2,356,864	473.9M	7.58 GB
level 4, rate 1/1024, 64-byte rewiring rows	1,248,384	231.2M	3.70 GB
+ known opened values as constants	1,137,536	226.8M	3.63 GB
+ one tensor for claims sharing a point	1,006,464	221.7M	3.55 GB
+ 200-bit Merkle digests	895,504	220.2M	3.52 GB
+ packed operand columns	834,832	219.2M	3.51 GB
+ linear forms, per-round folds, deep preprocessed cap	643,688	215.8M	3.45 GB
+ one ring switch per pair, eq-factored bus rounds	603,112	216.5M	3.46 GB
+ affine preprocessed forms, one shared forms sumcheck	574,952	217.8M	3.48 GB
+ preprocessed cap 18 layers deep	**564,552**	218.0M	3.49 GB

Table 11: Narrow recursion removes the statement's width from the garbled verifier's input. Exact counts for Ziren's binary stage and for the verifier of its narrow-recursion proofs (level 2 with 26-bit addresses, the smallest that fit the real tape); the rows below the third use the experimental changes listed below.

both packings behind a selector variable; (ii) Merkle digests are Blake3 truncated to 200 bits, which keeps collisions at 2^{100} work classically and $2^{66.7}$ under Brassard–Høyer–Tapp quantum collision search [BHT98]; (iii) each table sends one value per linear form of its columns that its constraints and buses read, derived from the symbolic AIR, and a sumcheck over the column index reduces them to the table's ring-switch claim, the verifier rebuilding the columns with a constant right inverse that costs only XORs; (iv) preprocessed forms are cut to those independent over the fixed trace's GF(2) column basis and the all-ones column, the rest are rebuilt by an affine inverse, and one sumcheck over the widest table's column index serves every table; and (v) bus product-GKR rounds send the round polynomial without its linear eq factor. Each change passes Plonky3's and Ziren's tests and its own tamper tests, but none has been reviewed; they are the Ziren and Plonky3 commits cited above.

Each proof meets 100 bits of composed soundness by Plonky3's assessment in the Johnson regime, with up to 46 bits of grinding per term at level 4, which halves under quantum search. Because the circuit checks only the last proof, the statement's soundness also composes over the earlier levels and over the zkVM proof itself. For the zkVM proof, Ziren reports more than 100 bits of interactive soundness per proof stage and 93.5 bits for an illustrative tree of 150 proofs in its measured Poseidon2 configuration; it gives no numerical bound for the non-interactive proof and claims no post-quantum security, because its cross-shard digest is an elliptic-curve multiset hash [DYH+26]. Proving level 4 alone took 1.8 hours on 64 cores with four GPUs grinding, and about 470 GB of memory.

At 564,552 bits, the input-publication fixtures of Table 3, with every transaction signed and every leaf executed under the stack limit, serialize to 1,205,217 vB for the 8-bit adaptor baseline (73 inputs in 13 reveal transactions), 4,263,180 vB for WOTS translation (481 inputs in 44), 6,425,757 vB for Antichain $(2, 16)$ (426 inputs in 71) and 9,363,254 vB for Lamport (566 inputs in 95, plus a 7,155 vB join); Disprove is one 97 vB transaction. Plain 4-bit Winternitz, modeled from one executed input at 23.06 weight units per bit, would be about 3.26 MvB, but it cannot deliver garbled labels without a translation table, which for WOTS translation costs 1.03 GB per instance at this input. The accounting boundary is that of Table 3.

Table 12 places the measured recursion verifier beside the estimates of [TF26b] for three zkVMs' recursion proofs.

verifier (hash)	non-free gates	garbled	input bits
RISC Zero 3.0.5 (Poseidon2)	15,640M	233.0 GiB	1.78M
RISC Zero 3.0.5 (Blake3)	1,560M	23.3 GiB	1.78M
SP1 6.3.1 shrink (Poseidon2)	28,200M	420.2 GiB	7.13M
SP1 6.3.1 shrink (Blake3)	2,810M	41.8 GiB	7.13M
Stwo `49f8e037` (Blake2s)	7,130M	106.2 GiB	4.37M
Stwo `49f8e037` (Blake3)	7,060M	105.2 GiB	4.37M
this work, narrow recursion (Blake3)	218.0M	3.25 GiB	0.56M

Table 12: Garbled verifiers of zkVM recursion proofs, at 16 bytes per non-free gate. The first six rows are the estimates of [TF26b], counted from the verifiers' operations, at the 96–100 classical bits their configurations report; their Blake3 rows assume the internal hash replaced with field arithmetic and proof size unchanged, and their input is the proof size. The last row is measured: the verifier of the last narrow-recursion proof (Table 11, 3.49 GB), whose run binds no public values and whose proof system carries the unreviewed changes of Appendix B.5.